

算法整理稿

算法竞赛知识点速查

目录

1	二叉堆与树状数组	4
1.1	树状数组	4
1.2	二维树状数组	5
2	字符串	6
2.1	字符串哈希	7
2.2	快速子串哈希	7
2.3	字典树	7
2.4	KMP 算法	9
3	线段树	10
3.1	线段树基础	10
3.2	可持久化线段树	13
4	有序表	20
4.1	平衡树	20
5	负环与差分约束	26
5.1	SPFA 求最短路	26
5.2	差分约束的两种形式	26
5.3	模板题: 洛谷 P5960 差分约束	27
6	倍增与 ST 表	30
6.1	ST 表构建	30
6.2	线段跳跃	31
6.3	RMQ 查询	31
7	背包问题优化	32
7.1	多重背包二进制优化	32
7.2	单调队列优化	33
8	树上 dp 问题	34
8.1	dfn 序	35

9 LCA-最近公共祖先	37
9.1 倍增 LCA	37
9.2 Tarjan 离线 LCA	38
10 博弈论	39
10.1 巴什博弈	39
10.2 尼姆博弈	40
11 树的性质	41
11.1 树的重心	41
11.2 树的直径	42
11.3 树上差分	45
12 数学	48
12.1 质因数分解 $O(\sqrt{N})$	48
12.2 素数筛埃氏筛	48
12.3 快速幂	49
12.4 矩阵快速幂	49
12.5 逆元	50
12.6 裴蜀定理	51
12.7 扩展欧几里得	52
12.8 pick 定理	53
13 线性基	53
13.1 异或线性基	53
13.2 向量线性基	55
13.3 CRT	56
13.4 EXCRT	56
14 高斯消元	57
14.1 加法方程组	57
14.2 异或方程组	59
15 一些小定理	59
16 同余最短路	61
17 快读	67
17.1 整数快读	67
17.2 防溢出乘法	67

18 康托展开、约瑟夫环	69
18.1 康托展开	69
18.2 逆康托展开	70
18.3 约瑟夫环问题	73

1 二叉堆与树状数组

1.1 树状数组

基本性质

常用于维护可差分信息

$O(\log n)$ 时间内实现单点修改、区间查询

数组 $tree[i]$ 表示其向前 $lowbit$ 长度的区间和

$lowbit(i)$ 表示 i 在二进制下最低位 1 代表的数

所需函数:

$lowbit$: 求出对应 $lowbit$ 值

```
int lowbit(int x) {  
    return x & (-x);  
}
```

建树: 利用前缀和预处理

```
for (int i = 1; i <= n; i++) {  
    int x;  
    cin >> x;  
    pre[i] = pre[i - 1] + x;  
    tree[i] = pre[i] - pre[i - lowbit(i)];  
}
```

$add(int\ x, int\ y)$: 将 x 位置的值加 y

```
void add(int x, int y) {  
    for (int i = x; i <= n; i += lowbit(i))  
        tree[i] += y;  
}
```

$sum(int\ x)$: 求 x 位置前缀和, 可利用这个求区间和

```
int sum(int x) {  
    int ans = 0;  
    for (int i = x; i >= 1; i -= lowbit(i))  
        ans += tree[i];  
    return ans;  
}
```

如需借助树状数组实现区间修改、单点查询, 则用树状数组维护原数组的差分数组即可, 即建树时无需进行前缀和预处理, 直接利用原数组建树, 其余操作遵循差分基本性质

1.2 二维树状数组

单点修改与范围查询

方法与一维基本相同, 所有一重循环变为二重循环, 插入初始数组时不需要前缀和, 直接调用 `add(x,y,z)` 增加

```
private int sum(int x, int y) {
    int ans = 0;
    for (int i = x; i > 0; i -= lowbit(i))
        for (int j = y; j > 0; j -= lowbit(j))
            ans += tree[i][j];
    return ans;
}
```

```
private void add(int x, int y, int v) {
    for (int i = x; i <= n; i += lowbit(i))
        for (int j = y; j <= m; j += lowbit(j))
            tree[i][j] += v;
}
```

范围修改与范围查询

核心公式:

`sum` 为原数组 $(1,1) \rightarrow (n,m)$ 的区间和

`D` 为原数组的差分数组

意义: 仅有 $x \geq i, y \geq j$ 的时候, (x,y) 这个格子才计入

后续推导:

所以需要四个树状数组来维护:

```
public static int[] [] info1 = new int[MAXN][MAXM]; // d[i][j]
public static int[] [] info2 = new int[MAXN][MAXM]; // d[i][j] * i
public static int[] [] info3 = new int[MAXN][MAXM]; // d[i][j] * j
public static int[] [] info4 = new int[MAXN][MAXM]; // d[i][j] * i * j
```

`add`:

```
public static void add(int x, int y, int v) {
    int v1 = v;
```

```

int v2 = x * v;
int v3 = y * v;
int v4 = x * y * v;
for (int i = x; i <= n; i += lowbit(i)) {
    for (int j = y; j <= m; j += lowbit(j)) {
        info1[i][j] += v1;
        info2[i][j] += v2;
        info3[i][j] += v3;
        info4[i][j] += v4;
    }
}
}

```

sum:

```

public static int sum(int x, int y) {
    int ans = 0;
    for (int i = x; i > 0; i -= lowbit(i)) {
        for (int j = y; j > 0; j -= lowbit(j)) {
            ans += (x + 1) * (y + 1) * info1[i][j]
                - (y + 1) * info2[i][j]
                - (x + 1) * info3[i][j]
                + info4[i][j];
        }
    }
    return ans;
}

```

初始化:

```

public static void add(int a, int b, int c, int d, int v) {
    add(a, b, v);
    add(a, d + 1, -v);
    add(c + 1, b, -v);
    add(c + 1, d + 1, v);
}

```

求区间和:

```

public static int range(int a, int b, int c, int d) {
    return sum(c, d) - sum(a - 1, d) - sum(c, b - 1) + sum(a - 1, b - 1);
}

```

2.1 字符串哈希

哈希值计算

对于一个字符串 `string s`, 其哈希值为 `hash`
 哈希方式:

```
unsigned long long hash = 0;
for (int i = 0; i < s.length(); i++) {
    hash = hash * base + (int)s[i];
}
```

通过定义哈希值类型为 `unsigned long long`, 实现 64 位溢出, 相当于对 2 的 64 次方取模
`base` 一般为 13131 或大于 300 的质数, 499 等

2.2 快速子串哈希

前缀哈希

核心思想: 前缀处理

`hash[i]` 表示字符串 `s` 前 `i` 位子串哈希值

需要预处理 `base` 的次方值数组 `pow[]`, 同样开 `ull` 类型以自动溢出

子串 `[l,r]` 哈希值为: $hash[l,r] = hash[r] - hash[l-1] * base$ 的 `r-l+1` 次方, `l=0` 时特殊处理

2.3 字典树

数组含义

需要依靠三个数组维护字典树:

```
tree[i][j]: i 为节点编号, j 表示第 j 条路径
```

`pass[i]`: 经过节点 `i` 的次数, 可用于求前缀出现次数

`end[i]`: 到达节点 `i` 结束的次数, 可用于求该字符串是否出现

基本操作

`getnum(char s)`: 将字符转化为代表路径的数字, `a->1/A->1/0->1`

```
int getnum(char s) {
    if (s >= '0' && s <= '9')
        return s - '0' + 1;
    if (s >= 'a' && s <= 'z')
        return s - 'a' + 11;
    if (s >= 'A' && s <= 'Z')
        return s - 'A' + 37;
    return 0;
}
```

`insert(string s)`: 将字符串插入字典树

```
void insert(string s) {
    int cur = 1;
    pass[cur]++;
    for (int i = 0; i < s.length(); i++) {
        int path = getnum(s[i]);
        if (tree[cur][path] == 0)
            tree[cur][path] = ++cnt;
        cur = tree[cur][path];
        pass[cur]++;
    }
    ed[cur]++;
}
```

`prefix(string s)`: 求以 `s` 为前缀的字符串数量

```
int prefix(string s) {
    int cur = 1;
    for (int i = 0; i < s.length(); i++) {
        int path = getnum(s[i]);
        if (tree[cur][path] == 0)
            return 0;
        cur = tree[cur][path];
    }
    return pass[cur];
}
```

`search(string s)`: 查看整体字符串 `s` 出现次数

```
int search(string s) {
    int cur = 1;
    for (int i = 0; i < s.length(); i++) {
        int path = getnum(s[i]);
        if (tree[cur][path] == 0)
            return 0;
        cur = tree[cur][path];
    }
    return ed[cur];
}
```

`del(string s)`: 删除字符串 `s`


```

void del(string s) {
    if (search(s)) {
        int cur = 1;
        pass[cur]--;
        for (int i = 0; i < s.length(); i++) {
            int path = getnum(s[i]);
            if (--pass[tree[cur][path]] == 0) {
                tree[cur][path] = 0;
                return;
            }
            cur = tree[cur][path];
        }
        ed[cur]--;
    }
}

```

典型应用

将数字转化为至多 32 位二进制数存储在字典树中, 用于快速处理异或和问题

洛谷 P4551: 最大异或路径

2.4 KMP 算法

用途与 next 数组

用途: 快速求解 s2 在 s1 中作为子串的第一个出现位置

next 数组: next[i]=k 表示 S2 的 i 位置之前的字符串 (不包含 s2[i] 也不包含整体), 其前缀与后缀能匹配的最大长度为 k, 同时也表示该最长前缀的下一个字符的所在位置为 k, 因为这个前缀是 0 到 k-1

KMP 字符串匹配过程示意图: 主串指针不回退, 模式串按 next 数组回跳。

构造与匹配

图解: 上图为不用跳的情况, 下图为需要跳的情况

KMP next 数组构造示意图: 相等则扩展最长相等前后缀, 失配则沿 next 继续回跳。

手绘示意图: KMP/next 的跳转过程, 展示失配后如何回跳到前缀位置。

```

void build() {
    nxt[0] = -1;
    nxt[1] = 0;
    int i = 2, cn = 0;
    while (i <= m) {
        if (s2[i - 1] == s2[cn]) {
            nxt[i++] = ++cn;
        } else if (cn > 0) {
            cn = nxt[cn];
        } else {

```

```

        nxt[i++] = 0;
    }
}

```

kmp 过程:

```

int kmp() {
    int x = 0, y = 0;
    while (x < n && y < m) {
        if (s1[x] == s2[y]) {
            x++;
            y++;
        } else if (y > 0) {
            y = nxt[y];
        } else {
            x++;
        }
    }
    return y == m ? x - y : -1;
}

```

循环节结论

对于一个总长为 n , 有 k 个循环节 (可以有剩余), 每个循环节长度为 L 的字符串, 其最大公共前后缀 ($\text{next}[n]$) 一定为 $(k-1)L + \text{剩余长度}$, 循环节长度 $L = kL + \text{剩余长度} - (k-1)L - \text{剩余长度} = n - \text{next}[n]$, 可以利用这个性质求得最小循环节长度

洛谷 P4391

3 线段树

3.1 线段树基础

基本性质与存储

作用: 高效区间修改、区间查询

性质: 对于一个长度为 n 的序列:

1. 对线段树上任一结点, 要么没孩子, 要么双孩子
2. 线段树只有 $2n-1$ 个结点, 高为 $\log n$
3. 结点编号最大不超过 $4n-1$

所需数据:

`tree[4*N]`: 用数组存储线段树

`tag[4*N]`: 懒标记数组

区间和模板

build: 数据输入完成后构建初始线段树

```
void build(int l, int r, int i) {
    if (l == r) {
        tree[i] = a[l];
        return;
    }
    int mid = (l + r) >> 1;
    build(l, mid, i << 1);
    build(mid + 1, r, i << 1 | 1);
    tree[i] = tree[i << 1] + tree[i << 1 | 1];
}
```

```
for (int i = 1; i <= n; i++)
    cin >> a[i];
build(1, n, 1);
```

lazy、down: 对懒标记操作, 组合使用

```
void lazy(int i, int v, int len) {
    tree[i] += v * len;
    tag[i] += v;
}
void down(int i, int ln, int rn) {
    if (tag[i] != 0) {
        lazy(i << 1, tag[i], ln);
        lazy(i << 1 | 1, tag[i], rn);
        tag[i] = 0;
    }
}
```

query: 查询区间和

```
int query(int job1, int job2, int l, int r, int i) {
    if (job1 <= l && job2 >= r)
        return tree[i];
    int mid = (l + r) >> 1;
    down(i, mid - l + 1, r - mid);
    int ans = 0;
    if (job1 <= mid)
        ans += query(job1, job2, l, mid, i << 1);
    if (job2 >= mid + 1)
        ans += query(job1, job2, mid + 1, r, i << 1 | 1);
    return ans;
}
```

add: 区间修改

```
void add(int jobl, int jobr, int jobv, int l, int r, int i) {
    if (jobl <= l && jobr >= r) {
        lazy(i, jobv, r - l + 1);
        return;
    }
    int mid = (l + r) >> 1;
    down(i, mid - l + 1, r - mid);
    if (jobl <= mid)
        add(jobl, jobr, jobv, l, mid, i << 1);
    if (jobr >= mid + 1)
        add(jobl, jobr, jobv, mid + 1, r, i << 1 | 1);
    tree[i] = tree[i << 1] + tree[i << 1 | 1];
}
```

线段树可以对区间执行不同的操作, 通过设计不同的懒标记与下传机制实现, 例如:

懒标记类型

对区间进行加减:tag 表示增减的数值

对区间进行重置:tag 表示重置为的值

对 01 进行反转:tag=0,1, 表示是否需要反转

对区间进行等差数列的加减:tag1 首项,tag2 公差

线段树可以维护不同类型的信息, 通过给线段树内存储的信息设计合适的类型、意义即可:

维护信息设计

维护区间最大值: 存储范围内的最大值

维护最大子段和: 洛谷 P4513 小白逛公园

存储四个信息: 区间左侧开头的最大子段和, 区间右侧开头的最大子段和, 区间最大子段和 (答案), 区间和

合并区间时, 区间和直接合并, 两侧开头最大子段和分别与另一区间全部和合并, 区间最大子段和由之前两侧答案、左侧区间右侧 + 右侧区间左侧合并取最大得到

```
void change(int loc, int v, int l, int r, int i) {
    if (l == r && loc == l) {
        sum[i] = v;
        ans[i] = v;
        lmax[i] = v;
        rmax[i] = v;
        return;
    }
    int mid = (l + r) >> 1;
    if (loc <= mid)
        change(loc, v, l, mid, i << 1);
    else
        change(loc, v, mid + 1, r, i << 1 | 1);
    sum[i] = sum[i << 1] + sum[i << 1 | 1];
    lmax[i] = max(lmax[i << 1], sum[i << 1] + lmax[i << 1 | 1]);
    rmax[i] = max(rmax[i << 1 | 1], sum[i << 1 | 1] + rmax[i << 1]);
    ans[i] = max(max(ans[i << 1], ans[i << 1 | 1]), rmax[i << 1] + lmax[i << 1 | 1]);
}
```

单点修改时, 无需维护懒标记

维护多种操作时需要考虑优先级, 即某一操作是否会影响另一操作, 不会被影响的优先, 如:

多标记优先级

1. 修改: 范围重置 + 范围增加查询: 区间和

进行范围重置时需要将懒标记中的范围增加取消, 懒标记下传时, 要先重置再增加

2. 修改: 范围增加 + 范围同乘查询: 区间和

获得乘法懒标记时需要同步扩大加法懒标记, 下传时先下传乘法再下传加法, 否则会导致多加一次

势能分析: 非经典线段树进行剪枝

势能分析剪枝

洛谷 P4145 线段树维护区间和, 修改操作为开根号

经典线段树无法在 $O(1)$ 时间内加工出一段区间内开根号后的区间和, 但是经过势能分析, 每个叶结点最多被开根号 6 次就会变为 1, 而对 1 开根号是没有意义的

基于这些性质, 可以在线段树上再维护一个区间最大值的信息, 当区间最大值为 1 时, 整棵子树就不需要再进行开方操作 (剪枝), 其查询总复杂度依旧为 $O(m \log n)$

此种线段树可以用于:

修改: 区间开根号维持: 区间和 + 区间最大值

当区间最大值为 1 时, 修改操作剪枝

修改: 区间取模 + 单点修改维持: 区间和

当区间最大值超过要求取模数时, 修改操作剪枝

```
void work(int jobl, int jobr, int jobv, int l, int r, int i) {
    if (maxx[i] < jobv)
        return; // 剪枝
    if (jobl <= l && jobr >= r && l == r) {
        tree[i] %= jobv;
        maxx[i] = tree[i];
        return;
    }
    int mid = (l + r) >> 1;
    if (jobl <= mid)
        work(jobl, jobr, jobv, l, mid, i << 1);
    if (jobr >= mid + 1)
        work(jobl, jobr, jobv, mid + 1, r, i << 1 | 1);
    tree[i] = tree[i << 1] + tree[i << 1 | 1];
    maxx[i] = max(maxx[i << 1], maxx[i << 1 | 1]);
}
```

3.2 可持久化线段树

单点修改主席树

单点修改的主席树:

可持久化线段树和标记永久化

可持久化线段树, 又叫主席树

做一次修改操作, 就生成一棵新版本线段树, 去处理比较复杂的区间查询问题

如果生成 n 个版本的线段树, 有 m 个查询操作, 那么单次生成、单次查询的时间复杂度为 $O(\log n)$

单点修改的可持久化线段树

- 1, 单点修改操作, 不需要懒更新机制
- 2, 新版本的线段树生成时, 沿途节点新建, 其他节点复用, 新建空间为 $O(\log n)$
- 3, 查询单点 x 的信息时, 根据版本号从根节点一路走到对应叶子即可
- 4, 查询区间 $[l, r]$ 的信息时, 可以利用 r 版本与 $(l-1)$ 版本做差
- 5, 总空间通常写作 $O(4n + n \log n)$

建立好初始版本线段树以后, 每次都建立新节点, 将对应老节点所有信息 (左右子节点、值) 全部复制过来, 然后看往那边走, 不被走的那一边就是被复用的节点

单点修改与单点查询的线段树模板:

模板题一

单点修改型可持久化线段树模板

给定一个长度为 n 的数组 arr , 下标范围为 $1 \sim n$, 原始数组视为 0 号版本

一共有 m 条操作, 每条操作属于以下两类之一

$v\ 1\ x\ y$: 基于第 v 号版本, 把位置 x 的值修改为 y , 并生成新版本

$v\ 2\ x$: 基于第 v 号版本, 查询位置 x 的值, 新版本与第 v 号版本一致

每条操作后得到的新版本数组, 版本编号为操作的计数

$1 \leq n, m \leq 10^6$

测试链接: <https://www.luogu.com.cn/problem/P3919>

```
const int maxn = 5e7 + 9;
int n, m;
int tree[maxn], root[maxn], lft[maxn], rgt[maxn], value[maxn];
int a[maxn];
int cnt = 0;
int build(int l, int r) {
    int rt = ++cnt;
    if (l == r) {
        value[rt] = a[l];
    } else {
        int mid = (l + r) >> 1;
        lft[rt] = build(l, mid);
        rgt[rt] = build(mid + 1, r);
    }
    return rt;
}
```

```

int update(int jobi, int jobv, int l, int r, int i) {
    int rt = ++cnt;
    lft[rt] = lft[i];
    rgt[rt] = rgt[i];
    value[rt] = value[i];
    if (l == r) {
        value[rt] = jobv;
    } else {
        int mid = (l + r) >> 1;
        if (jobi <= mid)
            lft[rt] = update(jobi, jobv, l, mid, lft[rt]);
        else
            rgt[rt] = update(jobi, jobv, mid + 1, r, rgt[rt]);
    }
    return rt;
}

int query(int jobi, int l, int r, int i) {
    if (l == r)
        return value[i];
    int mid = (l + r) >> 1;
    if (jobi <= mid)
        return query(jobi, l, mid, lft[i]);
    return query(jobi, mid + 1, r, rgt[i]);
}

```

```

cin >> n >> m;
for (int i = 1; i <= n; i++)
    cin >> a[i];
root[0] = build(1, n);
for (int i = 1; i <= m; i++) {
    int v, op, p;
    cin >> v >> op >> p;
    if (op == 1) {
        int x;
        cin >> x;
        root[i] = update(p, x, 1, n, root[v]);
    } else {
        root[i] = root[v];
        cout << query(p, 1, n, root[v]) << endl;
    }
}

```

第 k 小主席树

模板题二

主席树求区间第 k 小模板

给定一个长度为 n 的数组 arr ，下标范围为 $1 \sim n$ ，一共有 m 次查询

每次查询给出 l, r, k ，要求输出 $arr[l..r]$ 中第 k 小的数字

$1 \leq n, m \leq 2 \times 10^5$

$1 \leq arr[i] \leq 10^9$

测试链接: <https://www.luogu.com.cn/problem/P3834>

该种线段树为值域线段树: 来到某个节点时, 维护所有值的出现次数, 所以值域太大的时候, 大概率是要进行离散化的

建树时, 每读入一个数就建立一个版本的线段树

查询时, 对范围边界的两个版本的线段树做差, 可以得到边界范围内数字的出现次数, 从而得到往哪边走

```
const int maxn = 1e7 + 9;
int n, m;
int tree[maxn], lft[maxn], rgh[maxn], root[maxn], siz[maxn];
int a[maxn], ori[maxn]; // ori 复制原数组, a 负责输入、排序、去重
int cnt = 0;
map<int, int> Map; // 离散化后快速查询数字对应下标
int build(int l, int r) {
    int rt = ++cnt;
    siz[rt] = 0;
    if (l < r) {
        int mid = (l + r) >> 1;
        lft[rt] = build(l, mid);
        rgh[rt] = build(mid + 1, r);
    }
    return rt;
}
```

```
int insert(int jobi, int l, int r, int i) {
    int rt = ++cnt;
    lft[rt] = lft[i];
    rgh[rt] = rgh[i];
    siz[rt] = siz[i] + 1;
    if (l < r) {
        int mid = (l + r) >> 1;
        if (jobi <= mid)
            lft[rt] = insert(jobi, l, mid, lft[rt]);
        else
            rgh[rt] = insert(jobi, mid + 1, r, rgh[rt]);
    }
    return rt;
}

int query(int jobk, int l, int r, int u, int v) {
    if (l == r)
        return l;
    int lsize = siz[lft[v]] - siz[lft[u]];
    int mid = (l + r) >> 1;
    if (lsize >= jobk)
        return query(jobk, l, mid, lft[u], lft[v]);
    return query(jobk - lsize, mid + 1, r, rgh[u], rgh[v]);
}
```



```

cin >> n >> m;
for (int i = 1; i <= n; i++) {
    cin >> a[i];
    ori[i] = a[i];
}
sort(a + 1, a + n + 1);
int nn = unique(a + 1, a + n + 1) - a - 1;
for (int i = 1; i <= nn; i++)
    Map[a[i]] = i;
root[0] = build(1, nn);
for (int i = 1; i <= n; i++) {
    int x = Map[ori[i]];
    root[i] = insert(x, 1, nn, root[i - 1]);
}
for (int i = 1; i <= m; i++) {
    int l, r, k;
    cin >> l >> r >> k;
    cout << a[query(k, 1, nn, root[l - 1], root[r])] << endl;
}

```

范围修改主席树

范围修改的可持久化线段树

经典的方式

- 1, 范围修改操作, 需要懒更新机制
- 2, 仿照单点修改的可持久化线段树
- 3, 每来到一个节点, 新建节点并且复制老节点的信息
- 4, 当前节点的懒更新下发时 (down 过程), 左右孩子也新建, 接收懒更新信息, 务必让老节点信息保持不变

生成新版本的线段树, 单次额外空间为 $O(\log n)$

只要有懒更新的下发, 必然新建节点, 所以生成新版本的线段树、执行查询操作, 都会增加空间占用

```

const int maxn = 3e7;
int n, m;
int a[maxn], tree[maxn], tag[maxn], lft[maxn], rgh[maxn], root[maxn];
int cnt = 0;
int clone(int i) {
    int rt = ++cnt;
    lft[rt] = lft[i];
    rgh[rt] = rgh[i];
    tree[rt] = tree[i];
    tag[rt] = tag[i];
    return rt;
}
void lazy(int i, int v, int len) {
    tree[i] += v * len;
    tag[i] += v;
}

```

```

void down(int i, int ln, int rn) {
    if (tag[i] != 0) {
        lft[i] = clone(lft[i]);
        rgh[i] = clone(rgh[i]);
        lazy(lft[i], tag[i], ln);
        lazy(rgh[i], tag[i], rn);
        tag[i] = 0;
    }
}

```

```

int build(int l, int r) {
    int rt = ++cnt;
    tag[rt] = 0;
    if (l == r) {
        tree[rt] = a[l];
    } else {
        int mid = (l + r) >> 1;
        lft[rt] = build(l, mid);
        rgh[rt] = build(mid + 1, r);
        tree[rt] = tree[lft[rt]] + tree[rgh[rt]];
    }
    return rt;
}

int add(int jobl, int jobr, int jobv, int l, int r, int i) {
    int rt = clone(i);
    if (jobl <= l && jobr >= r) {
        lazy(rt, jobv, r - l + 1);
    } else {
        int mid = (l + r) >> 1;
        down(rt, mid - l + 1, r - mid);
        if (jobl <= mid)
            lft[rt] = add(jobl, jobr, jobv, l, mid, lft[rt]);
        if (jobr >= mid + 1)
            rgh[rt] = add(jobl, jobr, jobv, mid + 1, r, rgh[rt]);
        tree[rt] = tree[lft[rt]] + tree[rgh[rt]];
    }
    return rt;
}

```

```

int query(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && jobr >= r)
        return tree[i];
    int mid = (l + r) >> 1;
    down(i, mid - l + 1, r - mid);
    int ans = 0;
    if (jobl <= mid)
        ans += query(jobl, jobr, l, mid, lft[i]);
    if (jobr >= mid + 1)
        ans += query(jobl, jobr, mid + 1, r, rgh[i]);
    return ans;
}

```

```

cin >> n >> m;
for (int i = 1; i <= n; i++)
    cin >> a[i];
root[0] = build(1, n);
int t = 0;
for (int i = 1; i <= m; i++) {
    char op;
    cin >> op;
    if (op == 'C') {
        int l, r, d;
        cin >> l >> r >> d;
        root[t + 1] = add(l, r, d, 1, n, root[t]);
        t++;
    } else if (op == 'Q') {
        int l, r;
        cin >> l >> r;
        cout << query(l, r, 1, n, root[t]) << endl;
    } else if (op == 'H') {
        int l, r, k;
        cin >> l >> r >> k;
        cout << query(l, r, 1, n, root[k]) << endl;
    } else if (op == 'B') {
        int k;
        cin >> k;
        t = k;
    }
}
}

```

标记永久化

标记永久化

懒更新不再下发，变成只属于某个范围的标记信息，上下级的标记之间，不再相互影响
 查询时，懒更新也不再下发，从上往下的过程中，维护标记的叠加信息，即可完成查询
 标记挂在父范围不下发，也能在后续访问时正确合并

标记永久化，并不是标记信息的值不再变化，而是上下级标记之间不再相互影响
 可持久化线段树，可以使用标记永久化可以减少空间占用，但是应用范围比较窄
 范围增加 + 查询累加和，这一类的线段树，修改和查询的性质都有可叠加性，可以标记永久化

这一类的可持久化线段树，出题时会刻意缩减可用空间，目的就是考察标记永久化
 范围重置、查询最大值/最小值，这一类的线段树，修改和查询的性质不具有可叠加性，就用经典的方式

一旦标记永久化，就没有了懒更新的下发，那么查询时就不再新建节点了

如果生成 n 个版本的线段树，并配合 m 次查询操作，总空间通常写作 $O(4n + n \log n)$

经典求区间和、操作区间累加的标记永久化:

```

public static void add(int jobl, int jobr, long jobv, int l, int r, int i) {
    int a = Math.max(jobl, l);
    int b = Math.min(jobr, r);
    sum[i] += jobv * (b - a + 1);
    if (jobl <= l && r <= jobr) {
        addTag[i] += jobv;
    } else {
        int mid = (l + r) / 2;
        if (jobl <= mid)
            add(jobl, jobr, jobv, l, mid, i << 1);
        if (jobr > mid)
            add(jobl, jobr, jobv, mid + 1, r, i << 1 | 1);
    }
}

```

```

public static long query(int jobl, int jobr, long addHistory, int l, int r, int i) {
    if (jobl <= l && r <= jobr)
        return sum[i] + addHistory * (r - l + 1);
    int mid = (l + r) >> 1;
    long ans = 0;
    if (jobl <= mid)
        ans += query(jobl, jobr, addHistory + addTag[i], l, mid, i << 1);
    if (jobr > mid)
        ans += query(jobl, jobr, addHistory + addTag[i], mid + 1, r, i << 1 | 1);
    return ans;
}

```

4 有序表

4.1 平衡树

模板题: 洛谷 P3369 普通平衡树

题目要求动态维护一个可重集合 M , 并支持以下操作:

1. 向 M 中插入一个数 x ;
2. 从 M 中删除一个数 x , 若有多个相同的数, 只删除一个;
3. 查询 M 中严格小于 x 的数有多少个, 并将答案加一;
4. 将 M 从小到大排列后, 查询排名为 x 的数;
5. 查询 M 中小于 x 且最大的数, 即 x 的前驱;
6. 查询 M 中大于 x 且最小的数, 即 x 的后继。

对于操作 3、5、6, 不保证当前集合中存在数 x ; 对于操作 4、5、6, 保证答案一定存在。

测试链接: <https://www.luogu.com.cn/problem/P3369>

AVL 树的核心思想

AVL 树是一棵满足以下条件的二叉搜索树:

1. 对任意节点, 左子树中的所有键值小于节点键值, 右子树中的所有键值大于节点键值;
2. 对任意节点, 左右子树高度之差不超过 1;
3. 相同键值不重复建立节点, 而是在节点中维护出现次数 ‘cont’。

二叉搜索树负责保证有序性, AVL 的旋转操作负责把树高维持在 $O(\log n)$ 。因此插入、删除、排名、第 k 小、前驱和后继都可以在 $O(\log n)$ 时间内完成。

静态数组存储

模板使用数组模拟节点, 适合竞赛中提前确定节点上限的场景。编号为 0 的节点作为空节点, 全局数组默认初始化为零, 因此 ‘siz[0]’ 和 ‘hi[0]’ 都可以直接参与维护。

```
key[i]: 节点 i 保存的键值
cont[i]: 键值 key[i] 在节点 i 中出现的次数
siz[i]: 以 i 为根的子树中元素总数, 包含重复元素
hi[i]: 以 i 为根的子树高度
ls[i], rs[i]: 左、右孩子编号
head: 当前整棵 AVL 树的根节点编号
cnt: 已经分配的最大节点编号
```

‘siz’ 统计的是元素数量而不是节点数量, 这是处理重复值、排名和第 k 小查询的关键。

高度维护与旋转

每次修改一个子树后, 都要先重新计算当前节点的信息, 再检查平衡性:

```
siz[i] = siz[ls[i]] + siz[rs[i]] + cont[i]
hi[i] = max(hi[ls[i]], hi[rs[i]]) + 1
```

失衡只可能出现在四种形态中:

1. LL 型: 左孩子的左子树过高, 对当前节点右旋;
2. RR 型: 右孩子的右子树过高, 对当前节点左旋;
3. LR 型: 左孩子的右子树过高, 先对左孩子左旋, 再对当前节点右旋;
4. RL 型: 右孩子的左子树过高, 先对右孩子右旋, 再对当前节点左旋。

插入节点后, 从插入位置向上回溯并维护; 删除节点后, 也要从受影响的位置向上逐层检查。删除时可能同时出现 LL 与 LR, 或者 RR 与 RL, 因此代码中按子树高度关系选择调整方式即可。

插入与删除

插入时按照二叉搜索树的比较规则递归向下。遇到相同键值只增加 ‘cont’, 不新建节点; 回溯时依次执行 ‘up’ 和 ‘maintain’。

删除时分为三种情况:

1. 当前节点出现次数大于一, 只减少 ‘cont’;
2. 当前节点至多只有一个孩子, 直接用非空孩子替换当前节点;
3. 当前节点有两个孩子, 用右子树中的最小节点, 即后继节点, 替换当前节点, 再从右子树中删除这个后继节点。

模板中的 ‘removemostleft’ 负责从一棵子树中摘除最左节点, 并在回溯过程中维护 AVL 平衡。替换节点时, 必须同时接管原节点的左右子树, 否则会丢失原有子树。

排名、第 k 小、前驱与后继

排名查询只统计严格小于 ‘num’ 的元素数量, 最后加一即可。第 k 小查询根据左子树大小和当前节点的 ‘cont’ 决定向左、在当前节点返回, 还是向右递归。

前驱和后继不要求 x 一定存在:

1. 若当前键值大于等于 x , 前驱只能在左子树中寻找;
2. 若当前键值小于 x , 当前键值是一个候选答案, 继续在右子树中寻找更大的合法值;
3. 后继的判断方向完全相反。

P3369 AVL 模板代码

下面的代码使用静态数组维护一棵带重复值计数的 AVL 树, 支持 P3369 的六种操作。所有递归函数返回调整后的子树根编号, 这是旋转能够改变子树根的关键。

```
#include <algorithm>
#include <iostream>
#include <cstdio>

using namespace std;

const int MAXN = 100005;

int cnt = 0, head = 0;
int key[MAXN], hi[MAXN], ls[MAXN], rs[MAXN];
int cont[MAXN], siz[MAXN];

// 重新计算节点 i 的子树大小和高度。
void up(int i) {
    siz[i] = siz[ls[i]] + siz[rs[i]] + cont[i];
    hi[i] = max(hi[ls[i]], hi[rs[i]]) + 1;
}

// 左旋: 当前节点 i 的右孩子 r 上升, i 成为 r 的左孩子。
int leftrotate(int i) {
    int r = rs[i];
    rs[i] = ls[r];
    ls[r] = i;
    up(i);
    up(r);
    return r;
}

// 右旋: 当前节点 i 的左孩子 l 上升, i 成为 l 的右孩子。
int rightrotate(int i) {
    int l = ls[i];
    ls[i] = rs[l];
    rs[l] = i;
    up(i);
    up(l);
    return l;
}

// 检查节点 i 是否失衡, 并返回调整后的子树根。
```

```

int maintain(int i) {
    int lh = hi[ls[i]];
    int rh = hi[rs[i]];

    // 左子树过高: LL 直接右旋, LR 先左旋左孩子再右旋。
    if (lh - rh > 1) {
        if (hi[ls[ls[i]]] >= hi[rs[ls[i]]])
            i = rightrotate(i);
        else {
            ls[i] = leftrotate(ls[i]);
            i = rightrotate(i);
        }
    }
    // 右子树过高: RR 直接左旋, RL 先右旋右孩子再左旋。
    else if (rh - lh > 1) {
        if (hi[rs[rs[i]]] >= hi[ls[rs[i]]])
            i = leftrotate(i);
        else {
            rs[i] = rightrotate(rs[i]);
            i = leftrotate(i);
        }
    }
    return i;
}

// 返回严格小于 num 的元素数量。
int getrank(int i, int num) {
    if (i == 0)
        return 0;
    if (key[i] >= num)
        return getrank(ls[i], num);
    return siz[ls[i]] + cont[i] + getrank(rs[i], num);
}

// 题目中的排名从 1 开始, 因此在“小于 num 的数量”基础上加一。
int getrank(int num) {
    return getrank(head, num) + 1;
}

// 删除以 i 为根的子树中的最左节点 mostleft, 并维护回溯路径。
int removemostleft(int i, int mostleft) {
    if (i == mostleft)
        return rs[i];

    ls[i] = removemostleft(ls[i], mostleft);
    up(i);
    return maintain(i);
}

// 向以 i 为根的 AVL 子树中插入 num, 返回调整后的根。
int add(int i, int num) {
    if (i == 0) {
        key[++cnt] = num;
        cont[cnt] = 1;
        siz[cnt] = 1;
        hi[cnt] = 1;
        return cnt;
    }
}

```

```

    if (key[i] == num)
        cont[i]++;
    else if (key[i] > num)
        ls[i] = add(ls[i], num);
    else
        rs[i] = add(rs[i], num);

    up(i);
    return maintain(i);
}

// 从以 i 为根的 AVL 子树中删除一个 num, 返回调整后的根。
int remove(int i, int num) {
    if (key[i] < num) {
        rs[i] = remove(rs[i], num);
    } else if (key[i] > num) {
        ls[i] = remove(ls[i], num);
    } else if (cont[i] > 1) {
        // 有重复值时只删除一个, 节点本身仍然保留。
        cont[i]--;
    } else {
        if (ls[i] == 0 && rs[i] == 0)
            return 0;
        if (ls[i] != 0 && rs[i] == 0)
            i = ls[i];
        else if (ls[i] == 0 && rs[i] != 0)
            i = rs[i];
        else {
            // 两个孩子都存在时, 用右子树最小节点作为后继。
            int mostleft = rs[i];
            while (ls[mostleft] != 0)
                mostleft = ls[mostleft];

            rs[i] = removemostleft(rs[i], mostleft);
            ls[mostleft] = ls[i];
            rs[mostleft] = rs[i];
            i = mostleft;
        }
    }

    up(i);
    return maintain(i);
}

// 查询以 i 为根的子树中的第 x 小元素。
int index(int i, int x) {
    if (siz[ls[i]] >= x)
        return index(ls[i], x);
    if (siz[ls[i]] + cont[i] < x)
        return index(rs[i], x - siz[ls[i]] - cont[i]);
    return key[i];
}

// 查询严格小于 num 的最大键值。
int pre(int i, int num) {
    if (i == 0)
        return -999999999;

```



```
    if (key[i] >= num)
        return pre(ls[i], num);
    return max(key[i], pre(rs[i], num));
}

// 查询严格大于 num 的最小键值。
int post(int i, int num) {
    if (i == 0)
        return 999999999;
    if (key[i] <= num)
        return post(rs[i], num);
    return min(key[i], post(ls[i], num));
}

void add(int num) {
    head = add(head, num);
}

void remove(int num) {
    // getrank(num) != getrank(num + 1) 等价于 num 至少出现一次。
    if (getrank(num) != getrank(num + 1))
        head = remove(head, num);
}

int index(int x) {
    return index(head, x);
}

int pre(int num) {
    return pre(head, num);
}

int post(int num) {
    return post(head, num);
}

int main() {
    int t;
    cin >> t;
    while (t--) {
        int op, x;
        cin >> op >> x;
        if (op == 1)
            add(x);
        else if (op == 2)
            remove(x);
        else if (op == 3)
            cout << getrank(x) << '\n';
        else if (op == 4)
            cout << index(x) << '\n';
        else if (op == 5)
            cout << pre(x) << '\n';
        else
            cout << post(x) << '\n';
    }
    return 0;
}
```

复杂度与使用注意

设当前集合中共有 n 个元素, AVL 树的高度为 $O(\log n)$ 。插入、删除、排名、第 k 小、前驱和后继的时间复杂度均为 $O(\log n)$, 空间复杂度为 $O(n)$ 。

1. 数组大小要覆盖所有实际插入过的不同键值节点; 删除节点后模板不会回收编号;
2. 'siz' 必须统计重复元素, 不能误写成节点数量;
3. 每次旋转后要先更新下降的旧根, 再更新上升的新根;
4. 插入、删除递归返回值都可能是新的子树根, 父节点必须接收返回值;
5. 前驱和后继的哨兵值应根据题目值域调整, 不要让哨兵值与合法答案冲突;

6. 若键值可能接近 'int' 上界, 删除检查中的 'num + 1' 需要改用更安全的存在性判断, 避免整数溢出。

5 负环与差分约束

5.1 SPFA 求最短路

SPFA 可以看作队列优化的 Bellman-Ford 算法, 适用于带负权边的有向图。数组 'dis[i]' 维护当前从源点到节点 i 的最短距离, 数组 'vis[i]' 表示节点 i 是否已经在队列中。每次取出一个节点 'u', 枚举所有出边 (u, v, w) , 若满足

$$dis[v] > dis[u] + w,$$

就用新的更短距离更新 'dis[v]', 并把还不在队列中的 'v' 加入队列。重复这一过程直到队列为空。

如果图中不存在从源点可达的负环, SPFA 结束时得到的 'dis' 就是最短路; 如果存在可达负环, 沿着负环不断绕行可以让路径长度无限减小, 最短路不存在。

SPFA 判断负环

按本章模板的统计方式, 源点的初始化入队不计入 'cnt'; 节点每次因松弛成功而被加入队列时执行 'cnt[v]++', 包括变量节点第一次从源点松弛进入队列的情况。当图中只有 n 个节点时, 不经过重复节点的路径最多包含 $n - 1$ 条边, 因此若某个节点的入队次数满足 'cnt[i] > n - 1', 就说明松弛链条中出现了环; 这个环只能是负环。

需要注意, 负环判断中的阈值取决于图中的节点总数。差分约束通常会增加编号为 0 的超级源点, 原来有 n 个变量, 加上超级源点后共有 $n + 1$ 个节点, 所以模板中使用 'cnt[i] > n' 判断负环。若源点的第一次入队也计数, 对应判断形式会整体加一, 实现时必须与计数方式保持一致。

5.2 差分约束的两种形式

差分约束系统由若干个只包含两个变量之差的不等式组成, 目标是判断所有不等式是否有解, 并在有解时给出任意一组解。常见形式如下:

1. 形式一: $X_i - X_j \leq C$, 判断不等式组是否有解, 有解时给出一组变量取值;
2. 形式二: $X_i - X_j \geq C$, 判断不等式组是否有解, 有解时给出一组变量取值。

两种形式可以相互转换。形式二可以改写为

$$X_i - X_j \geq C \iff X_j - X_i \leq -C,$$

因此竞赛中通常统一转成形式一, 使用最短路判断负环。若直接把形式二看成最长路, 则对应判断的是正环; 两种说法本质相同。

差分约束与最短路的对应

将形式一改写为

$$X_i \leq X_j + C.$$

这恰好对应一条从 j 指向 i 、边权为 C 的有向边。若输入一条约束

$$X_x - X_y \leq z,$$

就建立边 $y \rightarrow x$, 边权为 z , 代码中的写法正是 `a[y].push_back({x, z})`。

沿着一个环把所有不等式相加, 左侧的变量会全部抵消。如果环的边权和小于 0, 就会得到形如

$$X_u \leq X_u - d \quad (d > 0),$$

这是不可能成立的, 所以负环意味着差分约束无解。反过来, 若不存在负环, 从一个能够到达所有节点的源点出发求出的最短距离满足每条边的松弛不等式, 因此可以作为差分约束的一组解。

超级源点与平移不变性

差分约束的原图不一定连通。为了让每个变量都能参与 SPFA, 新建超级源点 0, 并加入 $0 \rightarrow i$ 、边权为 0 的边。这样所有节点初始都可达, 且不会额外限制变量之间的差值。

若最短距离为 $(ans_1, ans_2, \dots, ans_n)$, 它就是一组可行解。由于约束只包含变量之差, 对任意常数 d 同时平移所有变量仍然满足约束:

$$(ans_1 + d, ans_2 + d, \dots, ans_n + d)$$

也是一组解, 所以有解时通常存在无穷多组解。使用超级源点后, 图中共有 $n+1$ 个节点, $m+n$ 条边, SPFA 的最坏时间复杂度仍按 $O(VE)$ 估计, 在本题记作 $O(nm)$; 空间复杂度为 $O(n+m)$ 。

若干变量确定的差分约束建图

如果有些变量直接确定了取值, 如何判断给定的不等式关系是否有解?

超级源点的设计

1. 连通超级源点, 例如 0 号点, 向所有节点连接权值为 0 的有向边, 保证图的连通性;
2. 限制超级源点, 例如 $n+1$ 号点。若 $X_i = a$, 从 $n+1$ 向 i 连接权值为 a 的边, 再从 i 向 $n+1$ 连接权值为 $-a$ 的边, 表示 $X_i = X_{n+1} + a$;
3. 利用 SPFA 判断负环, 如果出现负环, 说明给定的差分约束与固定值之间存在矛盾;
4. 连通超级源点和限制超级源点一定要分离。前者只负责保证节点可达, 后者只负责表示固定值。

5.3 模板题: 洛谷 P5960 差分约束

题目描述

给出一组包含 m 个不等式、含有 n 个未知数的不等式组：

$$\begin{cases} X_{c_1} - X_{c'_1} \leq y_1, \\ X_{c_2} - X_{c'_2} \leq y_2, \\ \vdots \\ X_{c_m} - X_{c'_m} \leq y_m. \end{cases}$$

求任意一组满足所有不等式的解。如果有多组解，输出其中任意一组；如果无解，输出 ‘NO’。

测试链接: <https://www.luogu.com.cn/problem/P5960>

输入格式

第一行输入两个正整数 n, m ，分别表示未知数的数量和不等式的数量。接下来 m 行，每行输入三个整数 c, c', y ，表示一个不等式

$$X_c - X_{c'} \leq y.$$

输出格式

若不等式组有解，输出一行 n 个整数，依次表示 $X_1, X_2, \dots, X_n$ 的一组可行解；若无解，输出 ‘NO’。

解题流程

1. 把每条 $X_c - X_{c'} \leq y$ 转化为边 $c' \rightarrow c$ ，边权为 y ；
2. 建立超级源点 0，向每个变量节点 i 连一条权值为 0 的边；
3. 以 0 为源点运行 SPFA，同时使用 ‘cnt’ 统计因松弛成功而入队的次数；
4. 若某个节点满足 ‘cnt[i] > n’，说明存在负环，不等式组无解，输出 ‘NO’；
5. 若 SPFA 正常结束，输出 ‘dis[1]’ 到 ‘dis[n]’，它们满足所有差分约束。

带注释的 SPFA 模板

下面的模板直接按照输入约束建图。因为加入了超级源点 0，代码中的 ‘n’ 表示原变量数量，负环阈值使用 ‘cnt[v] > n’。

```
#include <iostream>
#include <cstdio>
#include <vector>
#include <queue>

using namespace std;

struct node {
    int v, w;
};

int n, m;
vector<node> a[50003];
int dis[50003], cnt[50003];
bool vis[50003];
```

```

queue<int> q;

// 从 u 出发运行 SPFA。返回 true 表示没有负环，
// 返回 false 表示发现负环。
bool spfa(int u) {
    vis[u] = true;
    dis[u] = 0;
    cnt[u] = 0;
    q.push(u);

    while (!q.empty()) {
        int now = q.front();
        q.pop();
        vis[now] = false;

        for (int i = 0; i < a[now].size(); i++) {
            int v = a[now][i].v;
            int w = a[now][i].w;

            // 松弛: dis[v] <= dis[now] + w。
            if (dis[v] > dis[now] + w) {
                dis[v] = dis[now] + w;

                if (!vis[v]) {
                    // 超级源点的初始化入队不计数，这里只统计松弛入队次数。
                    cnt[v]++;

                    // 原变量有 n 个，加超级源点后共有 n + 1 个节点。
                    if (cnt[v] > n)
                        return false;

                    vis[v] = true;
                    q.push(v);
                }
            }
        }
    }
    return true;
}

int main() {
    cin >> n >> m;

    for (int i = 1; i <= m; i++) {
        int x, y, z;
        cin >> x >> y >> z;

        // 输入 x y z 表示  $X_x - X_y \leq z$ ,
        // 即  $dis[x] \leq dis[y] + z$ , 所以建边  $y \rightarrow x$ 。
        a[y].push_back({x, z});
    }

    // 超级源点 0 向所有变量连零权边，保证全部节点可达。
    for (int i = 1; i <= n; i++) {
        a[0].push_back({i, 0});
        dis[i] = 999999999;
    }
}

```

```

    if (!spfa(0))
        cout << "NO" << endl;
    else {
        for (int i = 1; i <= n; i++)
            cout << dis[i] << " ";
    }
    return 0;
}

```

模板要点

1. `a[y].push_back({x, z})` 是差分约束建图的核心, 方向写反会使得到的距离不再对应原不等式;
2. 超级源点的零权边只负责让所有变量可达, 不会改变变量之间的约束关系;
3. ‘cnt’ 的判断阈值必须和节点总数、初始入队是否计数保持一致。本模板初始入队不计数, 原变量数量为 n , 所以判断 ‘`cnt[i] > n`’;
4. ‘dis’ 既是 SPFA 求出的最短距离, 也是差分约束的一组可行解。若检测到负环, 则不存在任何可行解。

6 倍增与 ST 表

核心思想与适用范围

核心思想: 预处理 2 的幂次方的区间长度的信息, 通过拼凑区间来得到其他区间长度的信息

常见应用: 线段跳跃、区间最值 RMQ、区间最大公因数、区间最小公倍数、区间接位或、区间接位与

核心数据: `st[i][p]`: 从 i 出发长度为 $1 \ll p$ 的区间存储的信息

核心状态转移方程: `st[i][p] = st[st[i][p-1]][p-1]`

st 表共有 $\text{power} = \log_2(n)$ 列

ST 表适用范围: 两个区间 A,B 中重叠的部分不影响区间 A+B 的答案, 能通过 A 区间答案和 B 区间答案简单加工得出 (可重复贡献问题), 如最大最小值、区间公约数、按位与按位或, 适用于 RMQ 问题

6.1 ST 表构建

1. 预处理长度为 $1 \ll 0$ 的区间信息

常规:

```

for (int i = 1; i <= n; i++)
    st[i][0] = a[i];

```

(洛谷 P4155 国旗计划)

```
//双指针构建初始 st 表
for (int i = 1, arrive = 1; i <= n; i++) {
    while (arrive + 1 <= n && a[arrive + 1].s <= a[i].e)
        arrive++;
    st[i][0] = arrive;
}
```

2. 通过方程填充 st 表剩余部分

```
//套公式构建整体 st 表
for (int p = 1; p <= power; p++) {
    for (int i = 1; i <= n; i++)
        st[i][p] = st[st[i][p - 1]][p - 1];
}
```

6.2 线段跳跃

将区间 $[l, r]$ 长度二进制拆分, 从 1 开始向后跳, 每次跳拆出来的一个二的幂次方长度, 就可以拼凑出整个区间

最少线段覆盖环问题: 国旗计划

从 $p = \text{power}$ 开始判断下一步到达的点是否在目标范围内, 不在则 $p--$, 在则累加答案并更新现在所在位置

```
int sum = 0;
int aim = a[i].s + m;
int cur = i;
for (int p = power; p >= 0; p--) {
    int nxt = st[cur][p];
    if (nxt != 0 && a[nxt].e < aim) {
        sum += 1 << p;
        cur = nxt;
    }
}
```

6.3 RMQ 查询

可以优化, 区间合并的过程允许重叠, 只需要找到两个长度为 $1 \ll k$ 的区间并集为 $[l, r]$ 即可, 令 k 为满足 $1 \ll k \leq L$ 的最大整数, 返回 $\max(f[l][k], f[r - (1 \ll k) + 1][k])$ 即可

```
for (int i = 1; i <= n; i++) {
    cin >> a[i];
    st1[i][0] = a[i];
    st2[i][0] = a[i];
}
for (int p = 1; p <= log2(n); p++) {
    for (int i = 1; i + (1 << p) - 1 <= n; i++) {
        st1[i][p] = max(st1[i][p - 1], st1[i + (1 << p) - 1][p - 1]);
    }
}
```

```

        st2[i][p] = min(st2[i][p - 1], st2[i + (1 << (p - 1))][p - 1]);
    }
}

```

```

for (int i = 1; i <= q; i++) {
    int x, y;
    cin >> x >> y;
    int k;
    if (x == y)
        k = 0;
    else
        k = log2(y - x + 1);
    int maxx = max(st1[x][k], st1[y - (1 << k) + 1][k]);
    int minn = min(st2[x][k], st2[y - (1 << k) + 1][k]);
    cout << maxx - minn << endl;
}

```

7 背包问题优化

7.1 多重背包二进制优化

对于一种物品: 单个重量 w , 体积 v , 可选取数量 c

普通解法为循环枚举该物品数量

二进制优化: 只需要枚举 $\log(c)$ 次

将这种物品从 c 个拆成:

重量 w , 体积 v , 重量 $2w$, 体积 $2v$, 重量 $4w$, 体积 $4v$ 重量不足二次幂的剩余部分也成为
一个物品

```

for (int i = 1; i <= n; i++) {
    int x, y, c;
    cin >> x >> y >> c;
    for (int k = 1; k <= c; k <= 1) {
        cnt++;
        v[cnt] = x * k;
        w[cnt] = y * k;
        c -= k;
    }
    if (c > 0) {
        cnt++;
        v[cnt] = x * c;
        w[cnt] = y * c;
    }
}

```

后续视为 01 背包即可

7.2 单调队列优化

余数分组

按照对重量取余的余数分组, 例如:

i 号物品 $w=5, v=3, c=4$;

如果要求 $dp[i][20]$, 则:

```
dp[i][20] = max(
    dp[i - 1][8] + 4 * w,
    dp[i - 1][11] + 3 * w,
    dp[i - 1][14] + 2 * w,
    dp[i - 1][17] + 1 * w,
    dp[i - 1][20] + 0 * w
);
```

状态转移变形

```
dp[i][j] = max(dp[i - 1][j - k * v] + k * w)
令 j1 = j - k * v
则有:
dp[i][j] = max(dp[i - 1][j1] + (j - j1) / v * w)
```

原理: $dp[i][j] = \max(dp[i-1][j-kv] + kw)$, 令 $j1 = j - kv$

```
dp[i][j] = max(dp[i - 1][j1] + (j - j1) / v * w)
dp[i][j] = j / v * w + max(dp[i - 1][j1] - j1 / v * w)
其中 max 内的部分可以放入单调队列维护
```

```
int dp[105][100005];
deque<int> q;
int value1(int i, int j) {
    return dp[i - 1][j] - j / v[i] * w[i];
}
int main() {
    cin >> n >> m;
    for (int i = 1; i <= n; i++)
        cin >> v[i] >> w[i] >> c[i];
    for (int i = 1; i <= n; i++) {
        for (int modd = 0; modd <= min(m, v[i] - 1); modd++) {
            q.clear(); // 不要忘记清空队列
            for (int j = modd; j <= m; j += v[i]) {
                while (!q.empty() && value1(i, q.back()) < value1(i, j))
                    q.pop_back();
```

```

        q.push_back(j);
        while (!q.empty() && q.front() <= j - (c[i] + 1) * v[i])
            q.pop_front();
        dp[i][j] = value1(i, q.front()) + j / v[i] * w[i];
    }
}
cout << dp[n][m];
return 0;
}

```

8 树上 dp 问题

树形 DP 套路

树形 DP 研究的是树结构上的状态转移问题。

树的根节点没有父节点，其余每个节点都恰好有一个父节点；从根到任意节点的简单路径唯一，这也是树形 DP 能够成立的重要基础。

与一般图上的动态规划相比，树上的依赖关系通常更加清晰，大多数题目都体现为“父节点的答案依赖于各个子节点返回的信息”。

依赖关系清晰并不意味着题目简单，关键仍在于状态设计是否完整、合并方式是否正确。

常见分析步骤如下：

- 1) 先明确父节点要求出答案时，需要从每棵子树获得哪些信息；
- 2) 把这些信息组织成递归函数的返回值或状态结构；
- 3) 递归求出每棵子树的完整信息；
- 4) 在父节点处合并所有子树信息，得到当前节点的状态并继续向上返回。

树上 DP 往往不会出现大量重复子问题，节点间依赖方向也十分明确，因此很多题目直接使用树形递归即可，不一定需要额外写记忆化数组。

洛谷 P1352 没有上司的舞会上司参加附属员工不参加

没有上司的舞会

这道题目的上司与员工的关系就是一个树形关系，每一次 dfs 都要传回两个信息：自己参加时，以自己为根的最大快乐度；自己不参加时，以自己为根的最大快乐度。自己参加时的最大快乐度为所有子树的根不参加时最大快乐度的和，自己不参加时的最大快乐度为所有子树的根可参加也可不参加取最大的和。

```

struct Info {
    int yes, no;
    Info(int x, int y) {
        yes = x;
        no = y;
    }
}

```

```
};
Info dfs(int now) {
    int yes = happy[now], no = 0;
    for (int i = 0; i < a[now].size(); i++) {
        Info nxt = dfs(a[now][i]);
        yes += nxt.no;
        no += max(nxt.yes, nxt.no);
    }
    return Info(yes, no);
}
```

8.1 dfn 序

dfn 序性质

dfn 序是指按照深度优先遍历顺序给树上节点依次编号后得到的序列。

若某个节点的 dfn 编号为 x ，其子树大小为 y ，则编号区间 $[x, x + y - 1]$ 恰好对应这棵子树中的全部节点。

因此，只要同时记录 dfn 编号与子树大小，就可以把“树上的一个子树”转化为“序列上的一个连续区间”，从而方便进行子树定位、祖先关系判断以及跨子树讨论。

在 DFS 中记录每个节点的 dfn 编号与 ‘siz’ 即可：

```
void dfs(int now) {
    dfn[now] = ++cnt;
    siz[dfn[now]] = 1;
    for (int i = 0; i < a[now].size(); i++) {
        int v = a[now][i];
        dfs(v);
        siz[dfn[now]] += siz[dfn[v]];
    }
}
```

树上背包

洛谷 P2014 选课

课程关系为树状，选先修课后才能选后续的课程，需要一个虚拟头结点 0 将森林连起来

一种直接的状态设计是：‘dfs(i, j, k)’ 表示以 i 为根的树中，只考虑前 j 棵子树，恰好选择 k 门课程时能够获得的最大学分。

状态转移分为两类：

1. 不选择第 j 个子树，‘ $dp[i][j][k] = dp[i][j-1][k]$ ’
2. 选择第 j 个子树：枚举在该子树中选择 s 门课程

```
dp[i][j][k] = max(dp[i][j-1][k-s], dp[v][size[v]][s])
```

```
int dfs(int i, int j, int k) {
    if (k == 0)
        return 0;
    if (j == 0)
        return s[i];
    if (dp[i][j][k] != -1)
        return dp[i][j][k];
    int ans = dfs(i, j - 1, k);
    int v = a[i][j - 1];
    for (int s = 1; s < k; s++)
        ans = max(ans, dfs(i, j - 1, k - s) + dfs(v, a[v].size(), s));
    return dp[i][j][k] = ans;
}
```

从上述转移可以看出：如果某个节点不被选择，那么它整棵子树都会被整体跳过。利用 dfn 序的连续区间性质，可以把这一过程优化成按 dfn 编号从大到小转移，时间复杂度降为 $O(nm)$ 。

```
void dfs(int now) {
    dfn[now] = ++cnt;
    siz[cnt] = 1;
    val[cnt] = s[now];
    for (int i = 0; i < a[now].size(); i++) {
        int v = a[now][i];
        dfs(v);
        siz[dfn[now]] += siz[dfn[v]];
    }
}
```

```
for (int i = n + 1; i >= 1; i--) {
    for (int j = 1; j <= m + 1; j++) {
        dp[i][j] = max(dp[i + siz[i]][j], dp[i + 1][j - 1] + val[i]);
    }
}
```

洛谷 P1273 有线电视网

有线电视网

树状结构，叶子结点有正收益，边权为负收益，问你访问最多多少个叶子结点可以保持不亏本

这道题同样属于“不选当前节点就整棵子树一起跳过”的类型，因此也可以借助 dfn 序把时间复杂度压缩到 $O(nm)$ 。

需要注意：无法到达的无解状态应初始化为负无穷。

```
int dfs(int now, int k) {
    if (k == 0)
        return 0;
    if (now > n)
        return -999999999;
    if (dp[now][k] != -1)
```

```

    return dp[now][k];
int ans = dfs(now + siz[now], k);
ans = max(ans, dfs(now + 1, k - leaf[now]) + val[now]);
return dp[now][k] = ans;
}

```

9 LCA-最近公共祖先

9.1 倍增 LCA

树上倍增算法: 在线逐个求解 LCA

节点数为 n , 询问数为 m

预处理时间复杂度 $O(n \log n)$

单次查询时间复杂度 $O(\log n)$

m 次查询时间复杂度 $O(m \log n)$

步骤:(求 a, b 的最近公共祖先)

1. 利用倍增, 让 a, b 先来到同一层
2. 利用倍增, 让 a, b 同时往上跳, 保证各自到达的节点不同, 只跳二的幂次方步
3. 最后再往上跳一步即可

预处理与查询

预处理的同时初始化每个节点跳 2^0 步能到达的节点 $st[u][0] = f$, 并且依此得出跳不同幂次方步能到达的节点 (因为是从根节点开始的 dfs, 所以一定能建完整)

```

void dfs(int u, int f) {
    if (u == s)
        deep[u] = 1;
    else
        deep[u] = deep[f] + 1;
    st[u][0] = f;
    for (int p = 1; (1 << p) <= deep[u]; p++)
        st[u][p] = st[st[u][p - 1]][p - 1];
    for (int i = 0; i < a[u].size(); i++) {
        if (a[u][i] == f)
            continue;
        dfs(a[u][i], u);
    }
}

```

LCA 查询过程如下:

```

int lca(int x, int y) {
    if (deep[x] < deep[y])
        swap(x, y);
    for (int p = log2(n); p >= 0; p--) {
        if (deep[st[x][p]] >= deep[y])

```

```

        x = st[x][p];
    }
    if (x == y)
        return x;
    for (int p = log2(n); p >= 0; p--) {
        if (st[x][p] != st[y][p]) {
            x = st[x][p];
            y = st[y][p];
        }
    }
    return st[x][0];
}

```

建树时如果题目没有保证输入边已经按父子方向给出，就应补成双向边，并在遍历时避免沿父边回跳。

```

cin >> n >> m >> s;
for (int i = 1; i <= n - 1; i++) {
    int x, y;
    cin >> x >> y;
    a[x].push_back(y);
    a[y].push_back(x);
}
dfs(s, 0);

```

9.2 Tarjan 离线 LCA

时间复杂度 $O(n+m)$

步骤:

1. 建立每个节点的问题列表: 要建双向边, 并且要记录每个边对应的原来问题的编号, 用来后续输出

2. 来到当前节点 cur , 令 $vis[cur] = true$

3. 遍历 cur 所有子树, 每颗子树遍历完后都和 cur 节点合并成一个集合, 令 cur 为集合头结点

4. 遍历完 cur 所有子树后, 处理 cur 的问题 $LCA(cur, x)$:

如果 $vis[x] = true$, $LCA = x$ 所在集合头结点

如果 $vis[x] = false$, 忽略这个问题

建立节点对应的问题列表:

```

struct node {
    int v, index;
};

```

```
for (int i = 1; i <= m; i++) {
    int x, y;
    cin >> x >> y;
    q[x].push_back({y, i});
    q[y].push_back({x, i});
}
```

tarjan 过程:tarjan(s,0)

```
void tarjan(int u, int f) {
    vis[u] = 1; // 当前节点标记为访问过
    for (int i = 0; i < a[u].size(); i++) {
        int v = a[u][i];
        if (v == f)
            continue;
        tarjan(v, u);
        fa[v] = u;
    }
    for (int i = 0; i < q[u].size(); i++) {
        int v = q[u][i].v;
        if (!vis[v])
            continue;
        ans[q[u][i].index] = find(v);
    }
}
```

常用结论

一棵树上的三个节点, 他们的两两最近公共祖先数量 ≤ 2

对于一颗每条边权值为 1 的树, 从头结点到指定节点的距离就是深度 $deep$, 两节点之间的最短路径长就是 $deep[u] + deep[v] - 2 * deep[LCA]$

10 博弈论

10.1 巴什博弈

一共有 n 个石头, 两个人, 每个人每次可以拿 1 到 m 个石头, 谁拿走最后一个石头谁赢

一种直接做法是记忆化搜索: 设 $dp[i]$ 表示当前剩余 i 个石头时先手是否必胜。边界为 $dp[0] = 0$, 表示无石可取时当前行动者失败。转移时枚举拿走 $1 \sim m$ 个石头, 只要存在一种选择能让对手进入必败态, 则当前状态就是必胜态。

```
int dfs(int n, int m) {
    if (n == 0)
        return 0;
    if (dp[n] != -1)
        return dp[n];
    for (int i = 1; i <= m; i++) {
        if (dfs(n - i, m) == 0)
            return dp[n] = 1;
    }
}
```

```

    }
    return dp[n] = 0;
}

```

结论：当 $n \bmod (m + 1) \neq 0$ 时先手必胜；当 $n \bmod (m + 1) = 0$ 时先手必败。原因在于先手可以通过第一次操作，把局面调整到 $m + 1$ 的倍数；之后无论后手拿走多少个，先手都补足到 $m + 1$ ，从而始终把“整轮拿满 $m + 1$ 个”的节奏控制在自己手里，最终让对手面对必败局面。

反巴什博弈

取到最后一个石子的人输，相当于取到倒数第二个石子的人赢： $(n-1) \% (m+1) == 0$ 则先手输，不为零则先手赢

拓展巴什博弈

每次可以取质数的自然数幂次方的石头，如 $2^0=1, 2^2=4, 3^1=3$ 个石头

结论：可取数量集合中虽然形式复杂，但由于 1, 2, 3, 4, 5 都可以被取到，而 6 不能被直接取到，因此局面会呈现出以 6 为周期的规律。若先手能够始终把石头数控制为 6 的倍数留给对手，那么对手无论如何操作，都无法继续维持这一性质，先手便可重复该策略直至获胜。

10.2 尼姆博弈

地上有 n 堆石子，每一堆都有 a_i 个石子，两个人每个人都可以任选一堆石子选择 $1 - a_i$ 个石子拿走，拿走最后一个石头的人获胜

核心结论：把所有石子堆大小做异或运算。异或和不为 0 时先手必胜，异或和为 0 时后手必胜。

尼姆博弈证明

核心思想是：先手面对异或和不为 0 的局面时，总能交还给后手一个异或和为 0 的局面；而后手面对异或和为 0 的局面时，无论怎样操作，都只能把局面重新变成异或和不为 0。如此反复，最后后手会先面对全零局面。

1. 后手交还先手：异或为零不管怎么取，形成的局面异或一定不为零

任意某堆石头取走任意个石子，必然会导致原石子数二进制表示下某一位 1 的消失，从而改变所有堆石头该二进制位下 1 的奇偶性，导致异或值上这里由 0 变成 1

2. 先手交还后手：一定能通过某种操作将异或不为零的局面变成异或为零的局面

选择总异或最高位的 1，石头堆中石子二进制表示下该位有 1 的都可以选，假设选的 a ，那么 a 就要变成 $a \text{ xor } \text{总}$ ，设为 b ， b 一定小于 a ，因为是选的最高位的 1，所以更高位异或 0 是不变的，而 b 这一位由 1 变成了 0，后面不用管， b 一定 $< a$

反尼姆博弈

在尼姆博弈的基础上，规则改为了谁拿走最后一个石头，谁输

分情况讨论如下：

1. 如果石头堆石子数量全为 1：根据奇偶性，奇数后手赢，偶数先手赢

2. 如果只有一堆 >1 ，其余均为 1：先手必胜，因为先手可以调整自己是否取完那一堆 >1 的石头，从而调整石头堆石子数量为 1 的个数的奇偶性

3. 在 2 的情况下，异或结果一定不为 0，因为最高位异或起来肯定为 1；由尼姆博弈，如果我面对异或结果不为 0 的局面，我一定可以交还一个异或结果为 0 的局面，对手只能交还给我一个异或

结果为 0 的局面, 这就会进入循环; 也就是说, 如果一开始的局面异或不为 0, 我先手就一定能最先碰到情况 2, 从而进入必胜状态

综上可得结论:

如果所有石头堆大小都为 1: 偶数堆时先手赢, 奇数堆时后手赢

如果整体异或和不为 0: 先手赢

如果整体异或和为 0: 后手赢

11 树的性质

11.1 树的重心

基本定义

树的重心有以下几种等价定义, 求出的点是相同的。

- 1, 以某个节点为根时, 最大子树的节点数最小, 那么这个节点是重心;
- 2, 以某个节点为根时, 每棵子树的节点数都不超过总结点数的一半, 那么这个节点是重心;
- 3, 以某个节点为根时, 所有节点走向该节点的总边数最少, 那么这个节点是重心。

补充性质

- 1, 一棵树最多有两个重心;
- 2, 如果一棵树有两个重心, 那么这两个重心一定相邻;
- 3, 如果在树上增加或删除一个叶节点, 新的重心最多只会沿树边移动一条边;
- 4, 如果把两棵树连接起来, 那么新树的重心一定落在原来两棵树重心之间的路径上;
- 5, 如果树上边权都为正, 那么无论边权如何分布, 所有节点走向重心的总距离和最小。

基于定义一的求法

下面使用第一条定义求树的重心。设 ‘siz[u]’ 表示以 u 为根的子树大小, ‘maxsub’ 表示删去当前节点后剩余各个连通块中最大的节点数。若某个节点的 ‘maxsub’ 更小, 就可以用它更新当前重心。

```
void dfs(int u, int f) {
    siz[u] = 1;

    int maxsub = 0;
    for (int i = 0; i < a[u].size(); i++) {
        int v = a[u][i];
        if (v == f)
            continue;
        dfs(v, u);
        siz[u] += siz[v];
        maxsub = max(maxsub, siz[v]);
    }
}
```

```

    maxsub = max(maxsub, n - siz[u]);
    if (best > maxsub) {
        best = maxsub;
        center = u;
        c1 = -1;
        ok = 0;
    } else if (best == maxsub && u < center) {
        c1 = center;
        center = u;
        ok = 1;
    }
}

```

其中 ‘center’ 表示当前记录到的重心，‘c1’ 表示可能存在的另一个重心。若只要求任意一个重心，保留 ‘center’ 即可；若题目要求同时处理两个重心，则需要额外记录第二个答案。

基于定义二的求法

也可以利用第二条定义做更直接的判定。先对每个节点求出以它为根时最大的连通块大小，记作 ‘ms[u]’；随后检查是否满足 $ms[u] \leq n/2$ ，满足条件的节点就是重心。

```

void dfs(int u, int f) {
    siz[u] = 1;
    int maxsub = 0;

    for (int i = 0; i < a[u].size(); i++) {
        int v = a[u][i];
        if (v == f)
            continue;
        dfs(v, u);
        siz[u] += siz[v];
        maxsub = max(maxsub, siz[v]);
    }

    maxsub = max(maxsub, n - siz[u]);
    ms[u] = maxsub;
}

```

```

for (int i = 1; i <= n; i++) {
    if (ms[i] <= n / 2)
        cout << i << " ";
}

```

11.2 树的直径

基本概念

树上距离最远的两个点所形成的路径，称为树的直径。直径既可以表示为这条路径的长度，也可以进一步记录路径两端点和路径上的所有节点。

树的直径常用两类求法：

- 1, 两次 DFS 或 BFS: 适用于边权非负的树。优点是既能得到直径长度, 也能通过 ‘last’ 数组还原直径路径; 缺点是需要遍历两遍树。
- 2, 树形 DP: 适用于所有树。优点是只需要遍历一遍树; 缺点是通常只能直接得到直径长度。

正边权树的结论

如果树上的边权都为正, 则有如下直径相关结论:

- 1, 如果一棵树存在多条直径, 那么这些直径一定拥有公共的中间部分, 可能是一个公共点, 也可能是一段公共路径;
- 2, 树上任意一点的最远点集合中, 至少存在一个点是某条直径的端点。

两次 DFS 求直径

第一次从任意点出发, 找到距离它最远的点 ‘start’; 第二次从 ‘start’ 出发, 找到距离 ‘start’ 最远的点 ‘end’, 此时 ‘dis[end]’ 就是直径长度。‘last’ 数组用于记录搜索树中的父节点, 从而可以还原直径路径。

```
#include <iostream>
#include <cstdio>
#include <vector>
#include <cstring>
using namespace std;

struct node {
    int v, w;
};

int n;
vector<node> a[500005];
int dis[500005], last[500005];

void dfs(int u, int f) {
    last[u] = f;
    for (int i = 0; i < a[u].size(); i++) {
        int v = a[u][i].v;
        int w = a[u][i].w;
        if (v == f)
            continue;
        dis[v] = dis[u] + w;
        dfs(v, u);
    }
}

int main() {
    cin >> n;
    for (int i = 1; i < n; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        a[u].push_back({v, w});
        a[v].push_back({u, w});
    }
}
```

```

    }

    int start = 1;
    dfs(start, 0);
    for (int i = 1; i <= n; i++) {
        if (dis[i] > dis[start])
            start = i;
    }

    int end = 1;
    memset(dis, 0, sizeof(dis));
    dfs(start, 0);
    for (int i = 1; i <= n; i++) {
        if (dis[i] > dis[end])
            end = i;
    }

    cout << dis[end];
    return 0;
}

```

树形 DP 求直径长度

设 ‘dis[u]’ 表示从 u 出发，只向其子树方向走能够得到的最大路径长度；设 ‘ans[u]’ 表示必须经过 u 的最大路径长度。更新时先递归处理子节点，再用两条向下路径拼出经过当前节点的候选直径。

```

#include <iostream>
#include <cstdio>
#include <vector>
#include <cstring>
using namespace std;

struct node {
    int v, w;
};

int n;
vector<node> a[500005];
int dis[500005], ans[500005];

void dfs(int u, int f) {
    for (int i = 0; i < a[u].size(); i++) {
        int v = a[u][i].v;
        int w = a[u][i].w;
        if (v == f)
            continue;
        dfs(v, u);
    }

    for (int i = 0; i < a[u].size(); i++) {
        int v = a[u][i].v;
        int w = a[u][i].w;
        if (v == f)
            continue;
    }
}

```

```

        ans[u] = max(ans[u], dis[u] + dis[v] + w);
        dis[u] = max(dis[u], dis[v] + w);
    }
}

int main() {
    cin >> n;
    for (int i = 1; i < n; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        a[u].push_back({v, w});
        a[v].push_back({u, w});
    }

    dfs(1, 0);

    int maxi = 1;
    for (int i = 1; i <= n; i++) {
        if (ans[i] > ans[maxi])
            maxi = i;
    }
    cout << ans[maxi] << endl;
    return 0;
}

```

11.3 树上差分

树上点差分

树上点差分用于处理“多次对一条路径上的所有点权同时加上某个值，最后统一统计所有修改后的点权”的问题。关键是先用倍增 LCA 或 Tarjan 离线算法快速求出路径两端点的最近公共祖先，再把路径修改转化成若干个端点标记。

假设要让 x 到 y 的路径上所有点权都增加 v ，令 lca 为 x, y 的最近公共祖先，则点差分标记为：

```

num[x] += v;
num[y] += v;
num[lca] -= v;
num[parent[lca]] -= v;

```

所有修改都标记完成后，从根节点做一次 DFS 汇总子树贡献。遍历到节点 u 时，先递归处理 u 的所有子节点，再把每个子节点的 ‘num’ 累加回 ‘num[u]’。最终 ‘num[u]’ 就表示所有路径修改生效后节点 u 的点权。

树上边差分

树上边差分用于处理“多次对一条路径上的所有边权同时加上某个值，最后统一统计所有修改后的边权”的问题。仍然先求路径两端点的最近公共祖先。

假设要让 x 到 y 的路径上所有边权都增加 v ，令 lca 为 x, y 的最近公共祖先，则边差分标记为：

```
num[x] += v;
num[y] += v;
num[lca] -= 2 * v;
```

最后同样从根节点 DFS 汇总。若边 e 从父节点 u 连向子节点 v ，则汇总完成前后可以把 ‘num[v]’ 作为这条边所承受的累积修改量，即 ‘weight[e] += num[v]’，随后再把 ‘num[v]’ 累加到 ‘num[u]’。

点差分模板

下面的模板使用倍增 LCA 处理路径端点，并统计所有路径经过每个点的次数最大值。若每条路径贡献不是 1，只需要把四处差分标记中的 ‘++’ 和 ‘-’ 改成对应的 ‘+= v’ 与 ‘-= v’。

```
#include <iostream>
#include <cstdio>
#include <vector>
#include <cmath>
using namespace std;

int n, k;
vector<int> a[50004];
int num[50004];
int deep[50004], st[50004][63];
int maxx = -1;

void dfs_lca(int u, int f) {
    if (u == 1)
        deep[u] = 1;
    else
        deep[u] = deep[f] + 1;
    st[u][0] = f;
    for (int p = 1; (1 << p) <= deep[u]; p++)
        st[u][p] = st[st[u][p-1]][p-1];
    for (int i = 0; i < a[u].size(); i++) {
        if (a[u][i] == f)
            continue;
        dfs_lca(a[u][i], u);
    }
}

int lca(int x, int y) {
    if (deep[x] < deep[y])
        swap(x, y);
    for (int p = log2(n); p >= 0; p--) {
        if (deep[st[x][p]] >= deep[y])
            x = st[x][p];
    }
    if (x == y)
        return x;
    for (int p = log2(n); p >= 0; p--) {
        if (st[x][p] != st[y][p]) {
            x = st[x][p];
            y = st[y][p];
        }
    }
}
```

```

        return st[x][0];
    }

    void dfs(int u, int f) {
        for (int i = 0; i < a[u].size(); i++) {
            if (a[u][i] == f)
                continue;
            dfs(a[u][i], u);
        }

        for (int i = 0; i < a[u].size(); i++) {
            if (a[u][i] == f)
                continue;
            num[u] += num[a[u][i]];
        }
        maxx = max(maxx, num[u]);
    }

    int main() {
        cin >> n >> k;
        for (int i = 1; i < n; i++) {
            int x, y;
            cin >> x >> y;
            a[x].push_back(y);
            a[y].push_back(x);
        }

        dfs_lca(1, 0);

        for (int i = 1; i <= k; i++) {
            int x, y;
            cin >> x >> y;
            int fa = lca(x, y);
            num[x]++;
            num[y]++;
            num[fa]--;
            num[st[fa][0]]--;
        }

        dfs(1, 0);
        cout << maxx << endl;
        return 0;
    }
}

```

边差分模板

边差分与点差分的主体流程基本相同，仍然是先用 LCA 做路径端点标记，再从根节点向上汇总。区别只在最后的 DFS：遍历父节点 u 的每条子边时，设这条边连向子节点 v ，则 ‘num[v]’ 就是这条边需要累加的差分结果。因此只需要在点差分模板的 ‘dfs’ 中，把 “只累加子树贡献” 改成 “先更新边权，再累加子树贡献”。

```

void dfs(int u, int f) {
    for (int i = 0; i < a[u].size(); i++) {
        int v = a[u][i].v;
        int w = a[u][i].w;
    }
}

```

```

        if (v == f)
            continue;
        dfs(v, u);
    }
    for (int i = 0; i < a[u].size(); i++) {
        int v = a[u][i].v;
        int w = a[u][i].w;
        if (v == f)
            continue;
        a[u][i].w += num[v];
        num[u] += num[v];
    }
}

```

12 数学

12.1 质因数分解 $O(\sqrt{N})$

试除法模板

```

void work(int x) {
    for (int i = 2; i * i <= x; i++) {
        if (x % i == 0) {
            cout << i << endl;
            while (x % i == 0)
                x /= i;
        }
    }
    if (x > 1)
        cout << x << endl;
}

```

12.2 素数筛埃氏筛

埃氏筛

```

for (int i = 2; i < n; i++) {
    if (!vis[i]) {
        cnt++;
        for (int j = i; (long long)j * i < n; j++)
            vis[j * i] = 1;
    }
}

```

欧拉筛


```

for (int i = 2; i < n; i++) {
    if (vis[i] == 0)
        prime[++cnt] = i;
    for (int j = 1; j <= cnt; j++) {
        if (i * prime[j] > n)
            break;
        vis[i * prime[j]] = 1;
        if (i % prime[j] == 0)
            break;
    }
}

```

12.3 快速幂

快速幂模板

```

int ksm(int y, int x) {
    if (x == 0)
        return 1;
    if (y == 0)
        return 0;
    int sum = 1;
    while (x) {
        if (x & 1)
            sum = sum * y % modd;
        y = y * y % modd;
        x >>= 1;
    }
    return sum % modd;
}

```

12.4 矩阵快速幂

矩阵乘法模板

ans 存储快速幂结果,a 在自增

```

void ksm() {
    for (int i = 1; i <= n; i++)
        ans[i][i] = 1ll;
    while (k) {
        if (k & 1)
            multi1();
        multi2();
        k >>= 1;
    }
}

```

```

void multi1() {
    int a1[105][105] = {0};
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            for (int c = 1; c <= n; c++) {
                a1[i][j] += ans[i][c] * a[c][j] % modd;
                a1[i][j] %= modd;
            }
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            ans[i][j] = a1[i][j];
}
void multi2() {
    int a1[105][105] = {0};
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            for (int c = 1; c <= n; c++) {
                a1[i][j] += a[i][c] * a[c][j] % modd;
                a1[i][j] %= modd;
            }
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            a[i][j] = a1[i][j];
}

```

快速幂/矩阵快速幂应用

递推式加速

快速幂、矩阵快速幂可以用来解决一维多阶的递推表达式快速求解, 如:

一维一阶 $a_n = a_{n-1} * 5, a_1 = 1$

可以推出 $a_n = 1 * pow(5, n - 1)$, pow 可以用快速幂求解

一维二阶斐波纳契数列 $f(n) = 1 * f(n-1) + 1 * f(n-2)$

可以推出 $\{f(n+1), f(n)\} = \{f(1), f(0)\} \cdot ksm(\{(1, 1), (1, 0)\}, n)$

变换矩阵第一列为对应系数, 后面通过代入得到

多维一阶同一维一阶, 但是 a_n 为一个一维向量, a_n 只依赖于 a_{n-1} , 可以分析得出一个矩阵使得 a_{n-1} 变成 a_n

12.5 逆元

除法取模

当模数为质数且 b 与 MOD 互质时, 可以先求 b 的逆元, 再把除法转化为乘法。

b 的逆元可以用 $b^{MOD-2} \bmod MOD$ 求出。

因此 $(a/b) \bmod MOD = (a \bmod MOD) \cdot inv(b) \bmod MOD$ 。

连续逆元

```
// 连续数字逆元
// inv[i] 表示 i 在 mod p 意义下的逆元
inv[1] = 1;
inv[i] = (int)(p - 1LL * inv[p % i] * (p / i) % p);

// 连续阶乘逆元
// inv[i] 表示 i! 在 mod modd 意义下的逆元
inv[n] = ksm(fac[n], modd - 2);
inv[i] = 1LL * inv[i + 1] * (i + 1) % modd;
```

```
fac[1] = 1;
for (int i = 2; i <= 5005; i++)
    fac[i] = 1ll * fac[i - 1] * i % modd;
ifac[5000] = ksm(fac[5000], modd - 2) % modd;
for (int i = 4999; i >= 1; i--)
    ifac[i] = 1ll * ifac[i + 1] * (i + 1) % modd;
```

```
int C(int n, int m) {
    int res = 1ll * fac[n] * ifac[m] % modd * ifac[n - m] % modd;
    return res;
}
```

12.6 裴蜀定理

裴蜀定理

裴蜀定理

如果 a 和 b 是不全为 0 的整数, 则一定存在整数 x, y , 使得 $ax + by = \gcd(a, b)$ 。

证明过程与扩展欧几里得算法的推导紧密相关, 理解这一结论有助于把握后续算法为何正确。

需要重点把握的理解是:

a 和 b 的最大公约数, 是所有形如 $ax + by$ 的整数中最小的正数。

常见推论包括:

- 1, 如果 a, b 互质, 当且仅当存在整数 x, y 使得 $ax + by = 1$;
- 2, 如果方程 $ax + by = c$ 有整数解, 那么 c 一定是 $\gcd(a, b)$ 的倍数;
- 3, 二元情形可以进一步推广到多元线性组合;
- 4, 一旦 $ax + by = c$ 存在一组整数解, 就一定存在无穷多组整数解。

12.7 扩展欧几里得

扩展欧几里得算法

对于方程 $ax + by = \gcd(a, b)$, 当 a 和 b 确定以后, $\gcd(a, b)$ 也唯一确定, 通常要求入参 a, b 非负。

扩展欧几里得算法可以同时求出 a, b 的最大公约数 d , 以及方程 $ax + by = d$ 的一组特解 x, y 。

从递归推导角度看, 它正是对裴蜀定理构造过程的算法化实现。

时间复杂度

扩展欧几里得算法理论时间复杂度为 $O(\log \min\{a, b\})$ 。

在竞赛与算法题中通常可以直接作为标准模板使用。

```
void exgcd(int a, int b) {
    if (b == 0) {
        gcd = a;
        x = 1;
        y = 0;
    } else {
        exgcd(b, a % b);
        int px = x;
        int py = y;
        x = py;
        y = px - (a / b) * py;
    }
}
```

扩展欧几里得的通解形式:

$$x = x_0 * (c/d) + (b/d) * n$$

$$y = y_0 * (c/d) - (a/d) * n$$

$$n \in \mathbb{Z}$$

求最小非负特解: 用龟速乘实现 $x_0 * (c/d) \% (b/d)$

扩展欧几里得求逆元: 要求 num 和 modd 互质

```
exgcd(num, modd);
x = (x % modd + modd) % modd;
```

格点与鞋带公式

对于平面上两个整数坐标点 (x_1, y_1) 、 (x_2, y_2) , 两点之间的整数点 (包括自己) 的个数为 $\gcd(|x_1 - x_2|, |y_1 - y_2|) + 1$

求多边形面积的鞋带公式

n 个点 (x_i, y_i) 按照顺时针/逆时针排列, 且相邻点连线构成简单多边形。记

$$(x_{n+1}, y_{n+1}) = (x_1, y_1),$$

则多边形面积为

$$S = \frac{1}{2} \left| \sum_{i=1}^n (x_i y_{i+1} - x_{i+1} y_i) \right|.$$

12.8 pick 定理

Pick 定理

多边形面积 = 内格点数 + 边格点数/2-1

其中，面积可以由鞋带公式计算，边界上的格点数可以由前述 gcd 结论计算；这里统计边界点时不再额外加 1。

13 线性基

13.1 异或线性基

线性基原理

线性基常指的是异或空间线性基，向量空间线性基是下期的内容

一堆数字能得到的非 0 异或和的结果，能被元素个数尽量少的集合，不多不少的全部得到

那么就说，这个元素个数尽量少的集合，是这一堆数字的异或空间线性基

这里关注的是“去重后的非零异或和能否全部表示出来”，而不是每一种异或和出现了多少次

如下几个结论是构建异或空间线性基的基础，需要重点理解

- 1，一堆数字中，任意的 a 和 b，用 $a \oplus b$ 的结果替代 a、b 中的一个数字，不会影响非 0 异或和的组成
- 3，一堆数字能否异或出 0，在求出异或空间线性基之后，需要被单独标记

普通消元求解线性基

普通消元

zero 表示原数据能否异或构造出 0 这个数据

bas 表示线性基

```
bool insert(int num) {
    for (int i = 63; i >= 0; i--) {
        if ((num >> i) & 1) {
            if (bas[i] == 0) {
                bas[i] = num;
                return true;
            }
            num ^= bas[i];
        }
    }
    return false;
}

void work() {
```

```

zero = false;
for (int i = 1; i <= n; i++) {
    if (!insert(a[i]))
        zero = true;
}
}

```

依据这一组基可以得出:

基的数量 t : 遍历一遍 `bas` 数组, 统计不为 0 的数据

可以构造出的数据的数量: 2^t 的 t 次方-1

最大异或和: 从 `bas[63]` 到 `bas[0]` 依次异或当前最大值 (初始为 0), 如果异或后的最大值更大, 那就更新, 否则保留

高斯消元求解线性基

高斯消元

一堆数字的异或空间线性基, 可能不只一组, 求解方式为普通消元或者高斯消元

普通消元: 得到某一组线性基, 进而知道: 线性基大小、异或和个数、异或和是否有 0、最大异或和等信息

普通消元得到的是任意一组合线性基, 适合处理大多数基础问题

高斯消元: 得到标准形式的异或空间线性基, 既能知道普通消元得到的信息, 还能知道第 k 小的异或和

高斯消元得到的是标准形线性基, 便于进一步做顺序统计和结构分析

一般题目只需要普通消元得到的一组线性基, 足够解题, 除非需要知道第 k 小的异或和, 才会用到高斯消元

两种方法的时间复杂度都为 $O(nm)$, 其中 n 是数字个数, m 是最大数字的位数

异或空间线性基的规模为 $O(m)$, 其中 m 是最大数字的位数

```

void gauss() {
    len = 1;
    for (int i = 62; i >= 0; i--) {
        for (int j = len; j <= n; j++) {
            if (bas[j] & (1ll << i)) {
                swap(bas[j], bas[len]);
                break;
            }
        }
        if (bas[len] & (1ll << i)) {
            for (int j = 1; j <= n; j++) {
                if (j != len && (bas[j] & (1ll << i)))
                    bas[j] ^= bas[len];
            }
            len++;
        }
    }
    len--;
    zero = (len == n ? 0 : 1);
}

```

第 k 小异或和

将所有的基从小到大排列, 最小的编号为 0

将 k 写成最低位编号为 0 的二进制形式, 第几位上有 1 则答案异或上编号为几的基

13.2 向量线性基

普通消元

向量空间线性基由一组线性无关向量组成。依次将每个向量在已有主元下消元; 若消元后仍有绝对值大于 ε 的分量, 则将其加入线性基, 否则它与已有向量线性相关。下面的模板求出向量组的秩。

```
#include <iostream>
#include <cmath>

using namespace std;

const int MAXN = 505;
const double eps = 1e-5;
int n, m, rank_;
double a[MAXN][MAXN];
int bas[MAXN];

bool insert(int i) {
    for (int col = 1; col <= m; col++) {
        if (fabs(a[i][col]) <= eps)
            continue;
        if (bas[col] == 0) {
            bas[col] = i;
            return true;
        }
        double rate = a[i][col] / a[bas[col]][col];
        for (int k = col; k <= m; k++)
            a[i][k] -= rate * a[bas[col]][k];
    }
    return false;
}

int main() {
    cin >> n >> m;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            cin >> a[i][j];

    for (int i = 1; i <= n; i++) {
        if (insert(i))
            rank_++;
    }
    cout << rank_ << endl;
    return 0;
}
```

其中 $rank_$ 就是向量组的秩。模板使用普通加减乘除, 适用于实数向量; 浮点数比较时应根据题目数据范围调整 ε 。

13.3 CRT

中国剩余定理 (CRT)

给出 n 个同余方程: $x \equiv r_i \pmod{m_i}$, 其中所有 m_i 两两互质, 求最小非负解 x 。

求解思路:

1. 令 $lcm = m_1 * m_2 * \dots * m_n$ 。
2. 对每个方程, 令 $a_i = lcm / m_i$ 。
3. 求 a_i 在模 m_i 意义下的逆元 inv_i 。
4. 每一项贡献为 $c_i = r_i * a_i * inv_i$ 。
5. 最终答案为 $ans = (c_1 + c_2 + \dots + c_n) \bmod lcm$ 。

```
// c_i = r_i * (lcm / m_i) * inv(lcm / m_i) mod lcm
// 注意中间结果可能溢出, 必要时使用龟速乘 multi。
for (int i = 1; i <= n; i++) {
    a[i] = lcm / m[i];
    exgcd(a[i], m[i]);
    inv[i] = (x % m[i] + m[i]) % m[i];
    c[i] = multi(multi(r[i], a[i], lcm), inv[i], lcm);
    ans = (ans + c[i]) % lcm;
}
```

这种方法要求模数 m_1, m_2, m_3 两两互质

13.4 EXCRT

扩展中国剩余定理

给出 n 个同余方程, 求满足同余方程的最小正数解 x , 所有 M_i 之间可能并不互质

求解过程的关键在于把多个同余方程逐步合并成一个新的同余方程

1. 补充初始模数 $m_0 = 1$, $lcm = 1$, $tail = 0$, 那么, $ans = lcm * x + tail$, 这必然成立
2. 当前来到模数 m_i , 余数 r_i , 新的方程 $ans = m_i * y + r_i$, 两个方程相减得到新表达式
3. $lcm * x + m_i * y = r_i - tail$, 记为 $ax + by = c$, 扩展欧几里得算法求解
4. 如果不存在解, 过程结束, 说明不存在这样的 ans 。如果存在解, 得到最小非负特解 x_0
5. 通解 $x = x_0 + (b/d) * n$, 带入 $ans = lcm * x + tail$
6. 得到 $ans = lcm * (b/d) * n + (lcm * x_0 + tail)$
7. 令 $lcm * (b/d)$ 记为 lcm' , 令 $(lcm * x_0 + tail) \% lcm'$ 记为 $tail'$
8. 表达式又是 $ans = lcm' * x + tail'$, 去往下一组方程, 继续迭代 ans
9. 直到所有方程计算完毕, 最终返回 $tail$ 就是答案


```

int excrt() {
    int tail = 0, lcm = 1;
    // 初始 ans = lcm * x + tail
    for (int i = 1; i <= n; i++) {
        int a = lcm;
        int b = m[i];
        int c = r[i] - tail;
        c = (c % b + b) % b;
        exgcd(a, b);
        if (c % gcd != 0)
            return -1; // 无解
        int x0 = multi(x, c / gcd, b / gcd);
        int tmp = lcm * (b / gcd);
        tail = (multi(x0, lcm, tmp) + tail) % tmp;
        lcm = tmp;
    }
    return tail;
}

```

```

int multi(int a, int b, int modd) {
    a = (a % modd + modd) % modd;
    b = (b % modd + modd) % modd;
    int ans = 0;
    while (b != 0) {
        if ((b & 1) != 0)
            ans = (ans + a) % modd;
        a = (a + a) % modd;
        b >>= 1;
    }
    return ans;
}

```

14 高斯消元

14.1 加法方程组

本节实现中矩阵元素通常使用 ‘double’ 存储，因此判断某个值是否为 0 时，不能直接写成相等判断，而应比较 ‘fabs(x) < eps’，常用取值如 ‘eps = 1e-6’。

高斯消元解决加法方程组

高斯消元流程通常以加法方程组为例说明，但处理其他线性方程组时思路类似

如果题目要求严格区分矛盾、多解、唯一解，就需要额外关注主元与自由元的判定
理解高斯消元的关键，在于把“矩阵变形”和“解空间结构”联系起来

- 1, 先把问题整理成标准方程组；若变量数与方程数不一致，需要补齐矩阵的维度信息
- 2, 执行消元时要能够正确区分矛盾、唯一解和多解三种情况
- 3, 理解矩阵形态与解之间的对应关系，特别是主元与自由元的含义

4. 矩阵最终形态所表达的含义要读清：主元变量可能依赖若干自由元，而自由元之间彼此独立

高斯消元的过程时间复杂度为 $O(n^3)$ ，其中 n 为扩充后的方程个数

流程如下：

1. 对第 i 列处理时，从尚未确定主元的行中选出 $|a[j][i]|$ 最大的一行，与第 i 行交换；
2. 如果换行后该列主元仍视为 0，说明这一列无法确定主元，直接跳过；
3. 对当前行做归一化处理，使主元 $a[i][i]$ 变成 1；
4. 用当前主元行消去其他各行在第 i 列上的系数；
5. 理想情况下，最终会得到接近行最简形的矩阵结构。

当方程组数量不足时，可以理解为补充全 0 行： $0x_1 + 0x_2 + \cdots + 0x_n = 0$

当表达式中缺少某些变量时，则应在对应位置补 0 系数，例如： $ax_1 + bx_2 + 0x_3 + 0x_4$

```
void work_swap(int x, int y) {
    for (int j = 1; j <= n + 1; j++)
        swap(a[x][j], a[y][j]);
}
void gauss() {
    for (int i = 1; i <= n; i++) {
        int maxi = i;
        for (int j = 1; j <= n; j++) {
            if (j < i && fabs(a[j][j]) >= eps)
                continue;
            if (fabs(a[j][i]) > fabs(a[maxi][i]))
                maxi = j;
        }
        work_swap(maxi, i);
        if (fabs(a[i][i]) < eps)
            continue;
        double tmp = a[i][i];
        for (int j = 1; j <= n + 1; j++)
            a[i][j] /= tmp;
        for (int j = 1; j <= n; j++) {
            if (i == j)
                continue;
            double rat = a[j][i] / a[i][i];
            for (int k = 1; k <= n + 1; k++)
                a[j][k] -= rat * a[i][k];
        }
    }
}
```

解的判定

方程组与未知数个数相同的情形：

如果系数矩阵中出现全零行，就需要检查对应常数项是否为零；若存在非零常数项则方程组矛盾、无解，否则说明存在自由元，因而是多解。

方程数多于未知数时：若有效行的主元齐全且全零行不冲突，则为唯一解；若存在冲突，则无解；若主元不齐全但无冲突，则为多解。

方程数少于未知数时：

此时不可能出现唯一解；无冲突则为多解，有冲突则为无解。

14.2 异或方程组

最后一层循环需要写成 ' $k \leq n + 1$ '，否则会漏掉增广列。

```
void gauss() {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (j < i && a[j][j] == 1)
                continue;
            if (a[j][i] == 1) {
                work_swap(i, j);
                break;
            }
        }
        if (a[i][i] == 1) {
            for (int j = 1; j <= n; j++) {
                if (i != j && a[j][i] == 1) {
                    for (int k = 1; k <= n + 1; k++)
                        a[j][k] ^= a[i][k];
                }
            }
        }
    }
}
```

15 一些小定理

序与图论结论

狄尔沃斯定理: 最长链长度等于反链个数的最小值

求个数时可以用贪心 + 二分的思路

对一个数组拆分成两半整体交换位置, 相当于循环左移/右移

最大生成树上两点路径之间的最小边权, 是原图中的最大瓶颈路径的瓶颈值; 树上的两点路径一定是原图中的最大瓶颈路径中的一条

做同余减法的时候需要先加 modd, 不然可能会变成负数

错排问题

将 n 个元素放入 n 个位置, 要求每个元素都不在原来的位置, 这样的排列称为错排, 错排的方案数记为 $f(n)$ 。

初值为 $f(0) = 1, f(1) = 0$, 递推式为

$$f(n) = (n - 1)(f(n - 1) + f(n - 2))$$

基础模板

```
int gcd(int x, int y) {
    if (y == 0)
        return x;
    return gcd(y, x % y);
}
```

最短路模板

```
#include <iostream>
#include <cstdio>
#include <queue>
#include <vector>
#define int long long
using namespace std;
struct edge {
    int v, w;
};
struct node {
    int u, dis;
    friend bool operator<(const node &x, const node &y) {
        return x.dis > y.dis;
    }
};
int n, m, S;
int dis[100005];
bool vis[100005];
vector<edge> a[100005];
priority_queue<node> q;
```

```
for (int i = 1; i <= m; i++) {
    int x, y, z;
    cin >> x >> y >> z;
    a[x].push_back({y, z});
}
dis[S] = 0;
q.push({S, 0});
while (!q.empty()) {
    node now = q.top();
    q.pop();
    int u = now.u;
    if (vis[u])
        continue;
    vis[u] = 1;
    for (int i = 0; i < a[u].size(); i++) {
        int v = a[u][i].v;
        int w = a[u][i].w;
        if (dis[v] > dis[u] + w) {
            dis[v] = dis[u] + w;
            q.push({v, dis[v]});
        }
    }
}
```

$x_1 + x_2 + \cdots + x_m = t$ 的非负整数解个数: $C(m+t-1, m-1)$

二维 vector 动态初始化

```
vector<vector<int>>> a;
int main() {
    int n;
    cin >> n;
    a.assign(n, vector<int>());
}
```

位运算, 包括异或等运算, 优先级非常低, 一定要加括号

位运算与枚举

一般情况下, 前缀异或为 $q_i = a_1 \oplus a_2 \oplus \cdots \oplus a_i$ 。当 $a_i = i$ 时,

$$q_i = 1 \oplus 2 \oplus 3 \oplus \cdots \oplus i$$

有如下规律:

$$i \bmod 4 == 1, q_i = 1$$

$$i \bmod 4 == 2, q_i = i + 1$$

$$i \bmod 4 == 3, q_i = 0$$

$$i \bmod 4 == 0, q_i = i$$

状压 dp 中枚举子集:

j 会按从大到小的顺序枚举出 $status$ 的所有非空子集, 常用写法如下:

```
for (int j = status; j > 0; j = (j - 1) & status) {
    // 使用子集 j
}
```

位运算判定

```
public static boolean is2(int num) {
    return (num & (num - 1)) == 0;
}
```

16 同余最短路

基本思想

同余最短路把“能否拼出某个数”转化为“到达某个余数的最小代价”。选一个正整数 x 作为基准数, 只保留余数 $0, 1, \dots, x-1$ 这 x 个状态。若当前余数为 i , 使用一个数 y 后会转移到

$$(i + y) \bmod x,$$

边权为 y 。如果到达余数 i 的最小数为 $dis[i]$ ，那么同一余数还可以通过反复加上基准数 x 得到

$$dis[i], dis[i] + x, dis[i] + 2x, \dots$$

因此，只要求出每个余数对应的最小值，就能统计一定范围内可以到达的数。

题目一：跳楼机

有一栋 h 层的楼，从第一层出发，可以向上移动 x 层、 y 层或 z 层，也可以回到第一层。求使用跳楼机可以到达多少层。

把第一层看作高度 0，需要统计区间 $[0, h-1]$ 中可以到达的高度。以 x 为模数建立 x 个节点，节点 i 表示当前高度模 x 的余数，加入两类边：

$i \rightarrow (i + y) \bmod x$ ，边权为 y ；

$i \rightarrow (i + z) \bmod x$ ，边权为 z 。

加 x 层不会改变余数，因此不必单独建边。求出从余数 0 到各个余数的最短距离后，若 $dis[i] \leq h-1$ ，该余数能贡献

$$\left\lfloor \frac{h-1-dis[i]}{x} \right\rfloor + 1$$

层。所有边权非负，可以使用 Dijkstra，时间复杂度为 $O(x \log x)$ ，空间复杂度为 $O(x)$ 。

```
#include <iostream>
#include <cstdio>
#include <vector>
#include <queue>

using namespace std;

#define int long long

struct node {
    int v, w;
};

struct node1 {
    int u, dis;
    friend bool operator < (const node1 &x, const node1 &y) {
        return x.dis > y.dis;
    }
};

int h;
int x, y, z;
vector<node> a[100005];
priority_queue<node1> q;
int dis[100005], vis[100005];

void dij() {
    dis[0] = 0;
    q.push({0, 0});
}
```

```

while (!q.empty()) {
    node1 now = q.top();
    q.pop();
    int u = now.u;
    if (vis[u])
        continue;
    vis[u] = 1;

    for (int i = 0; i < a[u].size(); i++) {
        int v = a[u][i].v;
        int w = a[u][i].w;
        if (dis[v] > dis[u] + w) {
            dis[v] = dis[u] + w;
            q.push({v, dis[v]});
        }
    }
}

signed main() {
    cin >> h >> x >> y >> z;

    for (int i = 0; i < x; i++) {
        int t1 = (i + y) % x;
        int t2 = (i + z) % x;
        a[i].push_back({t1, y});
        a[i].push_back({t2, z});
        dis[i] = 1LL << 62;
    }

    dij();

    int ans = 0;
    for (int i = 0; i < x; i++) {
        if (dis[i] <= h - 1)
            ans += (h - 1 - dis[i]) / x + 1;
    }
    cout << ans << endl;
    return 0;
}

```

题目二：墨墨的等式

墨墨突然对等式很感兴趣，他正在研究

$$\sum_{i=1}^n a_i x_i = b$$

存在非负整数解的条件。他要你编写一个程序，给定 $n, a_1, \dots, a_n, l, r$ ，求出多少个 $b \in [l, r]$ 可以使等式存在非负整数解。

输入格式

第一行三个整数 n, l, r 。

第二行 n 个整数 $a_1, \dots, a_n$ 。

输出格式

一行一个整数，表示有多少个 $b \in [l, r]$ 可以使等式存在非负整数解。

输入输出样例

```
输入：
2 5 10
3 5

输出：
5
```

说明/提示

对于 20% 的数据， $n \leq 5, r \leq 10$ 。

对于 40% 的数据， $n \leq 10, r \leq 10^6$ 。

对于 100% 的数据， $n \leq 12, 0 \leq a_i \leq 5 \times 10^5, 1 \leq l \leq r \leq 10^{12}$ 。

两次转圈法

排序并去掉 0 后，令最小的正数 a_1 为基准数 x 。状态 i 表示当前数模 x 的余数， $\text{dis}[i]$ 表示得到余数 i 的最小值。初始时只有 $\text{dis}[0] = 0$ ，其余为无穷大。

处理数 a_i 时，余数转移为

$$\text{cur} \rightarrow (\text{cur} + a_i) \bmod x$$

这个转移会形成 $d = \gcd(a_i, x)$ 个环，分别从 $j = 0, 1, \dots, d-1$ 开始遍历。每个环上的节点都会被访问两圈：第一圈让已有的最短距离向前传播，第二圈让第一圈后面产生的更新绕回起点并继续传播。两圈后，本轮使用 a_i 任意次得到的最小值就都已经更新到 dis 中。

处理完所有数后，若余数 i 的最小值为 $\text{dis}[i]$ ，那么同一余数的其他可表示数为

$$\text{dis}[i], \text{dis}[i] + x, \text{dis}[i] + 2x, \dots$$

因此对于一个上界 t ，余数 i 的贡献为

$$\max\left(0, \left\lfloor \frac{t - \text{dis}[i]}{x} \right\rfloor + 1\right)$$

答案就是上界为 r 的总数减去上界为 $l-1$ 的总数。排序和去掉零值的时间复杂度为 $O(n \log n + nx)$ ，空间复杂度为 $O(x)$ 。

```
#include <iostream>
#include <cstdio>
#include <vector>
#include <algorithm>

using namespace std;

#define int long long
#define endl '\n'
```



```

int n, l, r;
int a[500005];
int dis[500005];

int gcd(int x, int y) {
    if (y == 0)
        return x;
    return gcd(y, x % y);
}

signed main() {
    cin >> n >> l >> r;
    l--;

    for (int i = 1; i <= n; i++)
        cin >> a[i];

    sort(a + 1, a + n + 1);
    int cnt = 0;
    for (int i = 1; i <= n; i++) {
        if (a[i] != 0)
            a[++cnt] = a[i];
    }

    if (cnt == 0) {
        cout << 0 << endl;
        return 0;
    }

    dis[0] = 0;
    int x = a[1];
    for (int i = 1; i < x; i++)
        dis[i] = 999999999999999;

    for (int i = 2; i <= cnt; i++) {
        int d = gcd(a[i], x);
        for (int j = 0; j < d; j++) {
            for (int cur = j, circle = 0;
                 circle < 2;
                 circle += cur == j ? 1 : 0) {
                int nxt = (cur + a[i]) % x;
                if (dis[cur] != 999999999999999)
                    dis[nxt] = min(dis[nxt], dis[cur] + a[i]);
                cur = nxt;
            }
        }
    }

    int ans = 0;
    for (int i = 0; i < x; i++) {
        if (r >= dis[i])
            ans += (r - dis[i]) / x + 1;
        if (l >= dis[i])
            ans -= (l - dis[i]) / x + 1;
    }
    cout << ans << endl;
    return 0;
}

```

```
}
```

两种方法的区别

Dijkstra方法直接把余数建成图，思路清晰，适合状态数 x 不太大的情况；两次转圈法不使用堆，利用加法取模后的环结构进行转移，适合题目允许 $O(nx)$ 的情况。两种方法的共同基础都是：先求每个余数能达到的最小数，再通过不断加上基准数统计答案。

17 快读

17.1 整数快读

```
inline int read() {
    int x = 0, f = 1;
    char ch = getchar();
    while (ch < '0' || ch > '9') {
        if (ch == '-')
            f = -1;
        ch = getchar();
    }
    while (ch >= '0' && ch <= '9') {
        x = x * 10 + ch - '0';
        ch = getchar();
    }
    return x * f;
}
```

17.2 防溢出乘法

针对取模乘法的防溢出乘法: 龟速乘

```
int add(int a, int b) {
    int ans = a;
    while (b != 0) {
        ans = a ^ b;
        b = (a & b) << 1;
        a = ans;
    }
    return ans % modd;
}

int multi(int a, int b) {
    int ans = 0;
    while (b != 0) {
        if ((b & 1) != 0)
            ans = add(ans, a);
        a = add(a, a);
        a %= modd;
        b >>= 1;
    }
    return ans % modd;
}
```

负数取模处理

也可以写成能够处理负数的形式:

```
int multi(int a, int b, int modd) {
    a = (a % modd + modd) % modd;
```

```
b = (b % modd + modd) % modd;
int ans = 0;
while (b != 0) {
    if ((b & 1) != 0)
        ans = (ans + a) % modd;
    a = (a + a) % modd;
    b >>= 1;
}
return ans;
}
```

18 康托展开、约瑟夫环

18.1 康托展开

题目

给定一个由 $1, 2, \dots, n$ 组成的排列, 求它在所有排列按字典序排列后的排名, 答案对 998244353 取模。

测试题目: 洛谷 P5367 康托展开。

排名公式

对于排列 S 的第 i 个位置, 记

$$c_i = |\{j \mid i < j \leq n, S_j < S_i\}|,$$

即 S_i 右侧比它小的数的个数。排列的排名从 0 开始时, 康托展开为

$$\text{rank}_0(S) = \sum_{i=1}^n c_i (n-i)!$$

题目通常要求从 1 开始的排名, 因此最终答案为 $\text{rank}_0(S) + 1$ 。

树状数组求解

从左到右扫描排列, 用树状数组维护还没有被使用的数字。开始时 $1, 2, \dots, n$ 都未使用; 处理 S_i 时, $\text{sum}(S[i]-1)$ 就是当前的 c_i , 再将 S_i 从树状数组中删除。

预处理阶乘 $\text{fac}[k] = k! \bmod 998244353$, 每一位的贡献为 $c_i \times \text{fac}[n-i]$ 。时间复杂度为 $O(n \log n)$, 空间复杂度为 $O(n)$ 。

```
#include <iostream>
#include <cstdio>

using namespace std;

#define int long long
#define endl '\n'

const int modd = 998244353;

int n;
int a[1000006];
int fac[1000006];
int tree[1000006];

int lowbit(int x) {
    return x & (-x);
}

void add(int x, int y) {
    for (int i = x; i <= n; i += lowbit(i))
        tree[i] += y;
}

int sum(int x) {
```

```

    int ans = 0;
    for (int i = x; i >= 1; i -= lowbit(i))
        ans += tree[i];
    return ans;
}

signed main() {
    cin >> n;
    for (int i = 1; i <= n; i++)
        cin >> a[i];

    fac[0] = 1;
    for (int i = 1; i <= n; i++)
        fac[i] = fac[i - 1] * i % modd;

    // 初始时 1..n 都还没有被使用。
    for (int i = 1; i <= n; i++)
        add(i, 1);

    int ans = 0;
    for (int i = 1; i <= n; i++) {
        // 统计当前仍未使用且小于 a[i] 的数。
        ans += sum(a[i] - 1) * fac[n - i] % modd;
        ans %= modd;
        // 当前元素已经处理，后续不再参与统计。
        add(a[i], -1);
    }

    cout << (ans + 1) % modd << endl;
    return 0;
}

```

18.2 逆康托展开

题目

给定一个由 $1, 2, \dots, n$ 组成的排列，设它当前的排名为 x ，求排名为 $x + m$ 的排列。题目保证 $x + m$ 不超过所有排列的总数。

数据范围： $1 \leq n \leq 10^5$ ， $1 \leq m \leq 10^{15}$ 。

测试题目：洛谷 U72177 逆康托展开。

阶乘进制转换

康托展开得到的排名可以写成

下面采用从 0 开始的排名表示；若题目中的排名从 1 开始，应先减 1。

$$x = c_1(n-1)! + c_2(n-2)! + \dots + c_{n-1}1! + c_n0!,$$

其中第 i 位 c_i 的取值范围是 $0 \leq c_i \leq n-i$ 。因此它不是普通的十进制，而是阶乘进制：从左到右各位的基数依次为 $n, n-1, \dots, 2, 1$ 。

为了求出排名增加 m 后的各位数字，先把 m 加到最低位，再从后往前进位。第 i 位的基数为

$$base_i = n - i + 1,$$

所以进位过程为

$$a[i-1] += \left\lfloor \frac{a[i]}{base_i} \right\rfloor, \quad a[i] \leftarrow a[i] \bmod base_i$$

代码中的

```
a[i - 1] += a[i] / (n - i + 1);
a[i] %= n - i + 1;
```

就是这个阶乘进制转换。特别地，最后一位的基数是 1，所以它不能保留数值，所有增加的量都会向前进位。例如 $n = 4$ 时，四位的基数依次为 4, 3, 2, 1。

线段树解码

先从左到右扫描原排列，用线段树维护还没有使用的数字。对于当前数字 a_i ，线段树查询区间 $[1, a_i - 1]$ 的和，就得到它对应的康托展开位；然后删除 a_i 。

完成阶乘进制加法后，重新建立一棵线段树。此时 a_i 表示当前应该选择剩余数字中第 $a_i + 1$ 小的数，使用线段树查询第 k 个仍未使用的数字并删除它，即可还原目标排列。时间复杂度为 $O(n \log n)$ ，空间复杂度为 $O(n)$ 。

```
#include <iostream>
#include <cstdio>

using namespace std;

#define int long long
#define endl '\n'

int n, m;
int a[100005];
int tree[400005];

void build(int l, int r, int i) {
    if (l == r) {
        tree[i] = 1;
        return;
    }

    int mid = l + r >> 1;
    build(l, mid, i << 1);
    build(mid + 1, r, i << 1 | 1);
    tree[i] = tree[i << 1] + tree[i << 1 | 1];
}

void add(int jobi, int jobv, int l, int r, int i) {
    if (l == r) {
        tree[i] += jobv;
        return;
    }
    int mid = l + r >> 1;
    if (mid >= jobi)
        add(jobi, jobv, l, mid, i << 1);
    else
```

```

        add(jobi, jobv, mid + 1, r, i << 1 | 1);
        tree[i] = tree[i << 1] + tree[i << 1 | 1];
    }

int getans(int k, int l, int r, int i) {
    if (l == r) {
        tree[i]--;
        return l;
    }
    int mid = l + r >> 1;
    int ans;
    if (tree[i << 1] >= k)
        ans = getans(k, l, mid, i << 1);
    else
        ans = getans(k - tree[i << 1], mid + 1, r, i << 1 | 1);
    tree[i] = tree[i << 1] + tree[i << 1 | 1];
    return ans;
}

int sum(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && jobr >= r)
        return tree[i];
    int mid = l + r >> 1;
    int ans = 0;
    if (jobl <= mid)
        ans += sum(jobl, jobr, l, mid, i << 1);
    if (jobr >= mid + 1)
        ans += sum(jobl, jobr, mid + 1, r, i << 1 | 1);
    return ans;
}

signed main() {
    cin >> n >> m;
    for (int i = 1; i <= n; i++)
        cin >> a[i];

    build(1, n, 1);
    for (int i = 1; i <= n; i++) {
        int x = a[i];
        // 统计当前仍未使用且小于 x 的数字，得到阶乘进制的一位。
        if (x == 1)
            a[i] = 0;
        else
            a[i] = sum(1, x - 1, 1, n, 1);
        add(x, -1, 1, n, 1);
    }

    // a[i] 是阶乘进制的第 i 位；先把 m 加到最低位。
    a[n] += m;
    for (int i = n; i >= 1; i--) {
        int base = n - i + 1;
        // 当前位只保留 [0, base - 1]，多出的部分进到前一位。
        a[i - 1] += a[i] / base;
        a[i] %= base;
    }

    build(1, n, 1);
    for (int i = 1; i <= n; i++)

```



```

    // a[i] 从 0 开始，因此要取剩余数字中的第 a[i] + 1 小。
    cout << getans(a[i] + 1, 1, n, 1) << " ";
    return 0;
}

```

18.3 约瑟夫环问题

题目

有 n 个首尾相接的节点，编号为 $1, 2, \dots, n$ 。从 1 号节点开始报数，报到 k 的节点被删除，之后从下一个节点重新从 1 开始报数，直到环上只剩一个节点，求该节点的原始编号。

数据范围： $1 \leq n, k \leq 10^6$ 。

测试题目：洛谷 P8671 约瑟夫环问题。

递推推导

每一轮游戏开始时，都把当前开始位置重新命名为 1。设上一轮序列长度为 n ，本轮序列长度为 $n - 1$ ，上一轮被杀节点在上一轮序列中的编号为 p 。若本轮序列中的节点编号为 new ，则它在上一轮序列中的原始编号为

$$\text{old} = (\text{new} + p - 1) \bmod n + 1.$$

这里公式中的 n 指上一轮序列的长度。被杀节点的编号与 k 的关系为

$$p = (k - 1) \bmod n + 1.$$

从最后一轮开始反推：最后只剩一个节点，重新编号后它必为 1。设待恢复的上一轮序列长度为 c ，把本轮得到的答案代入上式，得到

$$\text{ans} = (\text{ans} + k - 1) \bmod c + 1.$$

令 c 从 2 递增到 n ，最终的 ans 就是原序列中的答案。时间复杂度为 $O(n)$ ，空间复杂度为 $O(1)$ 。

```

#include <iostream>
#include <cstdio>

using namespace std;

int main() {
    int n, k;
    cin >> n >> k;

    int ans = 1; // 只有一个节点时，答案为 1。
    for (int c = 2; c <= n; c++) {
        // c 是当前待恢复的上一轮序列长度。
        ans = (ans + k - 1) % c + 1;
    }

    cout << ans << endl;
    return 0;
}

```